

Pico RGB LED

From Waveshare Wiki

Jump to: navigation, search

Overview

The Pico-RGB-LED is an LED matrix module designed for Raspberry Pi Pico, which has 160 (16×10) RGB full-color external-controlled constant current integrated LEDs, with embedded quality controller IC and LED chip. It is small in size, uses very few peripheral components, and presents a clean-looking front panel. Controlled by a Raspberry Pi Pico, this LED panel is programable for a high-level, colorful matrix display.

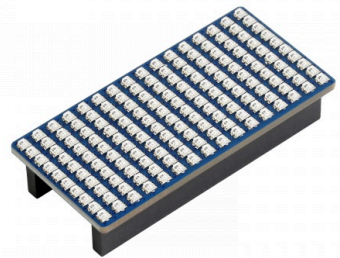
Feature

- Standard Raspberry Pi Pico header, supports Raspberry Pi Pico series.
- 160 RGB LEDs arranged in a 16×10 grid, multiple IO selection.
- Right-angled 2.54mm pitch male pin header on the bottom, for easy extension.
- 0807 colorful LEDs, integrating quality single wire cascading constant current driver IC and quality RGB LED chip, aka external-controlled constant *current integrated LEDs.
- Embedded controller IC, with features like high reliability, high anti-interference performance, high constant current accuracy, and low power consumption.
- Integrated quality LED chip, with high consistent brightness and pure white light effect, lower light degradation.

Specifications

- Operating voltage: 5V
- Communication: Single wire return to zero code
- Color depth: 24-bit (each 8-bit of RGB)
- Data rate: 800Kbps
- Brightness: adjustable 256 level
- Pixels: 16×10 pixels

Pico RGB LED

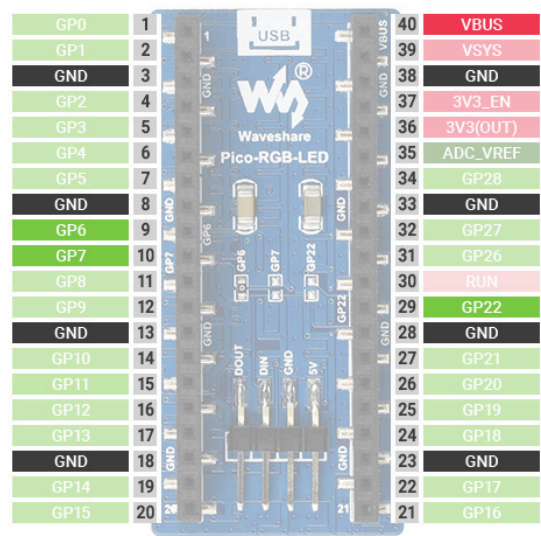


(<https://www.waveshare.com/pico-rgb-led.htm>)

RGB Full-color LED Matrix Panel for
Raspberry Pi Pico, 16×10 Grid

- Dimensions: 54.00 x 26.00mm

Pinout

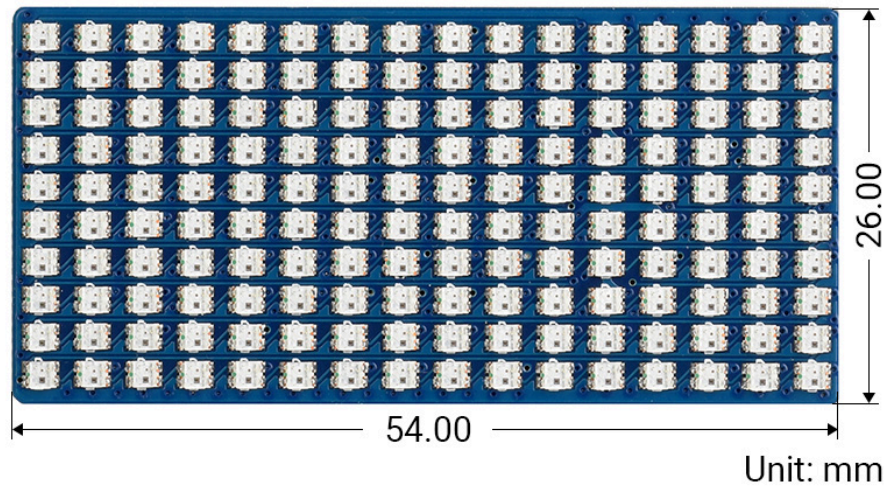


VBUS		Power input
GND		Ground
GP6	DIN	Data input (default)
GP7	DIN	Data input (alternate)
GP22	DIN	Data input (alternate)

(/wiki/File:Pico-

RGB-LED-details-inter.jpg)

Dimension



(/wiki/File:Pico-

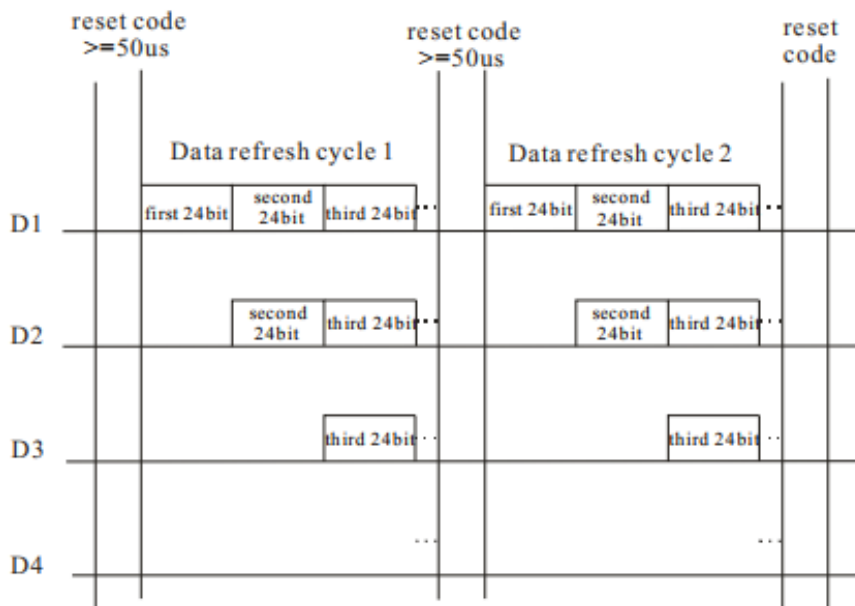
RGB-LED-details-size.jpg)

About the RGB LED

Lamp bead used single communication way, with the method of zero code signal. After the chip is powered on and reset, it receives the data from the DIN end. After receiving enough 24 bits, the DO port begins to forward data for the next chip to provide input data. Before forwarding, the DO port is lowered. At this time, the lamp will not receive new data, the built-in RGB chip according to the received 24-bit data after the different duty cycle signal, show different brightness. If the DIN input signal is a RESET signal, the chip will send the received data to display, the chip will receive new data after the end of the signal, after receiving the starting 24-bit data, data will be forwarded through the DO port before the lamp does not receive the RESET code, the RGB brightness remains unchanged.

When the low-level RESET code above 200us is received, the RGB chip inside the lamp bead will show different brightness according to the different duty cycle signals generated after the 24-bit data just received.

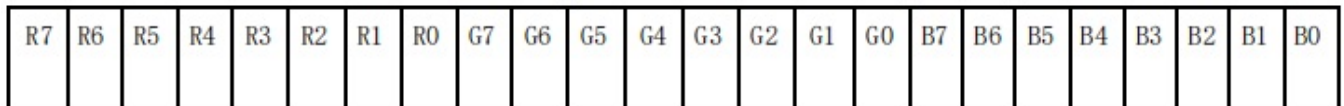
Data Transmission



(/wiki/File:Rgb_led_hat_senddata.png)

Note: D1 is the data sent from MCU and D2, D3, D4 are the data automatically shaped and forwarded by cascaded circuits.

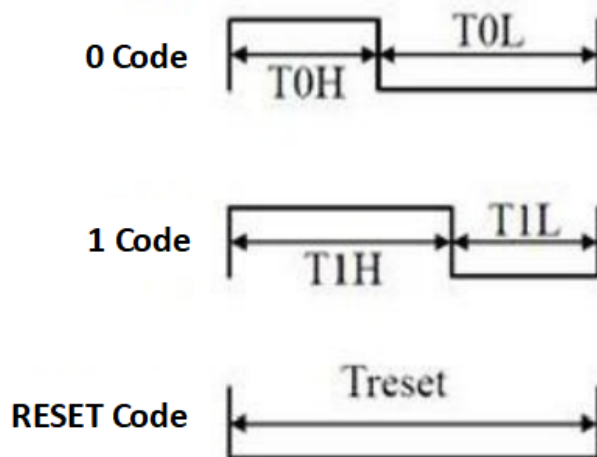
24-bit data structure



(/wiki/File:Pico-RGB-LED-24bit.jpg)

Note: Data in high bits are sent first and follow the RGB order.

Timing



(/wiki/File:Pico-RGB-LED-SX02.png)

T0H	0 code, time of High signal	0.35μs	±150ns
T1H	1 code, time of High signal	1.36μs	±150ns
T0L	0 code, time of Low signal	1.36μs	±150ns
T1L	1 code, time of Low signal	0.35μs	±150ns
RES	RESET code	50μs	\

Notice

- The product in RGB full light (24 for 1), will lead to the lamp bead surface temperature being too high, it is strictly prohibited to feel the temperature by hand.
- The current of this product is close to 2.5A when RGB is fully lit. It is recommended to use A power supply of 5V,3A, or more.
- Since the USB output current of the computer is low, we set the brightness data at RGB555, even if the full light is only about 300mA.
- Also do not set the current too low, if your brightness data is less than 4 bits, part of the light will not be bright.

Hardware Connection

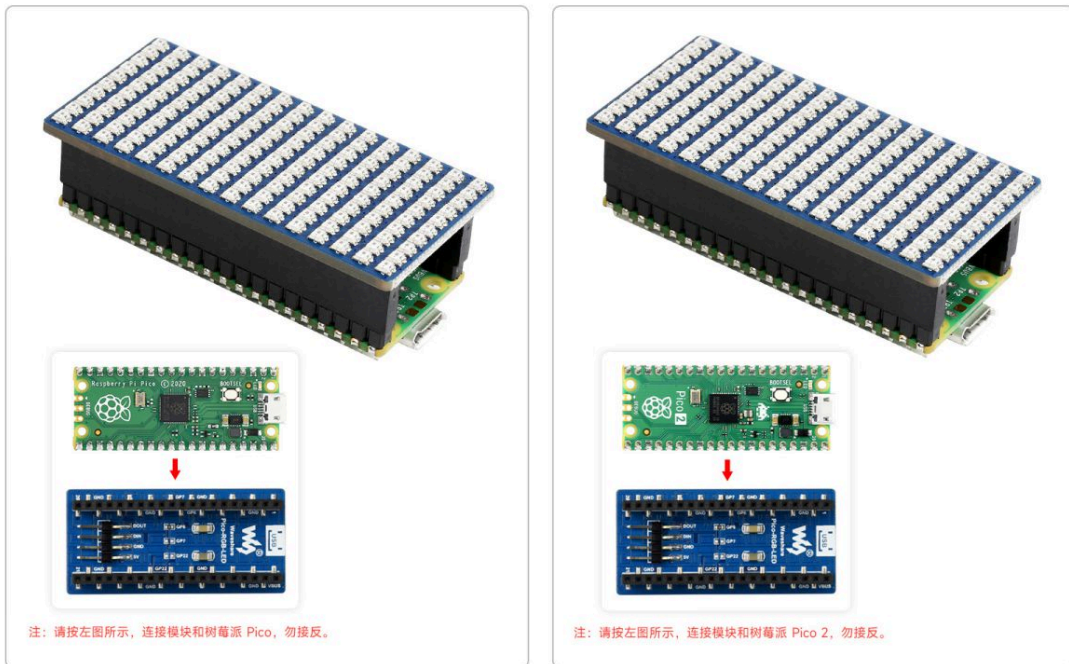
Please take care of the direction when you connect Pico, an USB port is printed to indicate. You can also check the pin of Pico and the LCD board when connecting.

You can connect the display according to the table.

RGB LED	Pico	Description
VCC	VBUS	Power Input
GND	GND	GND
DIN	GP6	Data input
DIN	GP7	Data input
DIN	GP22	Data input

DOUT	\	Data output
------	---	-------------

Connection



(/wiki/File:Pico-

RGB-LED-connect.jpg)

Setup Environment

Please refer to Raspberry Pi's guide: <https://www.raspberrypi.org/documentation/pico/getting-started/>
(<https://www.raspberrypi.org/documentation/pico/getting-started/>)

Download Demo codes

```
sudo apt-get install p7zip-full
cd ~
sudo wget https://files.waveshare.com/upload/5/5a/Pico-RGB-LED_code.7z
7z x Pico-RGB-LED_code.7z -o./Pico-RGB-LED_code
cd ~/Pico-RGB-LED_code
cd c/build/
```

Run the Demo codes

This guide is based on Raspberry Pi.

C Examples

Open a terminal and enter the directory of C codes:

```
cd ~/Pico-RGB-LED_code/c/
```

Create a build folder and add SDK:

For example, if the path of SDK is ../../pico-sdk

Then you should create a build and add the path like these:

```
mkdir build
cd build
export PICO_SDK_PATH=../../pico-sdk
```

Run cmake.. command to generate the Makefile file:

```
#Pico
cmake -DPICO_BOARD=pico -DPICO_PLATFORM=rp2040 ..
#Pico2
cmake -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 ..
```

Run make command to build.

```
make -j9
```

After the compilation is complete, the uf2 file will be generated. Press and hold the button on the Pico board, connect the Pico to the USB port of the Raspberry Pi through the Micro USB cable, and release the button. After connecting, the Raspberry Pi will automatically recognize a removable disk (RPI-RP2) and copy the main.uf2 file in the build folder to the recognized removable disk (RPI-RP2).

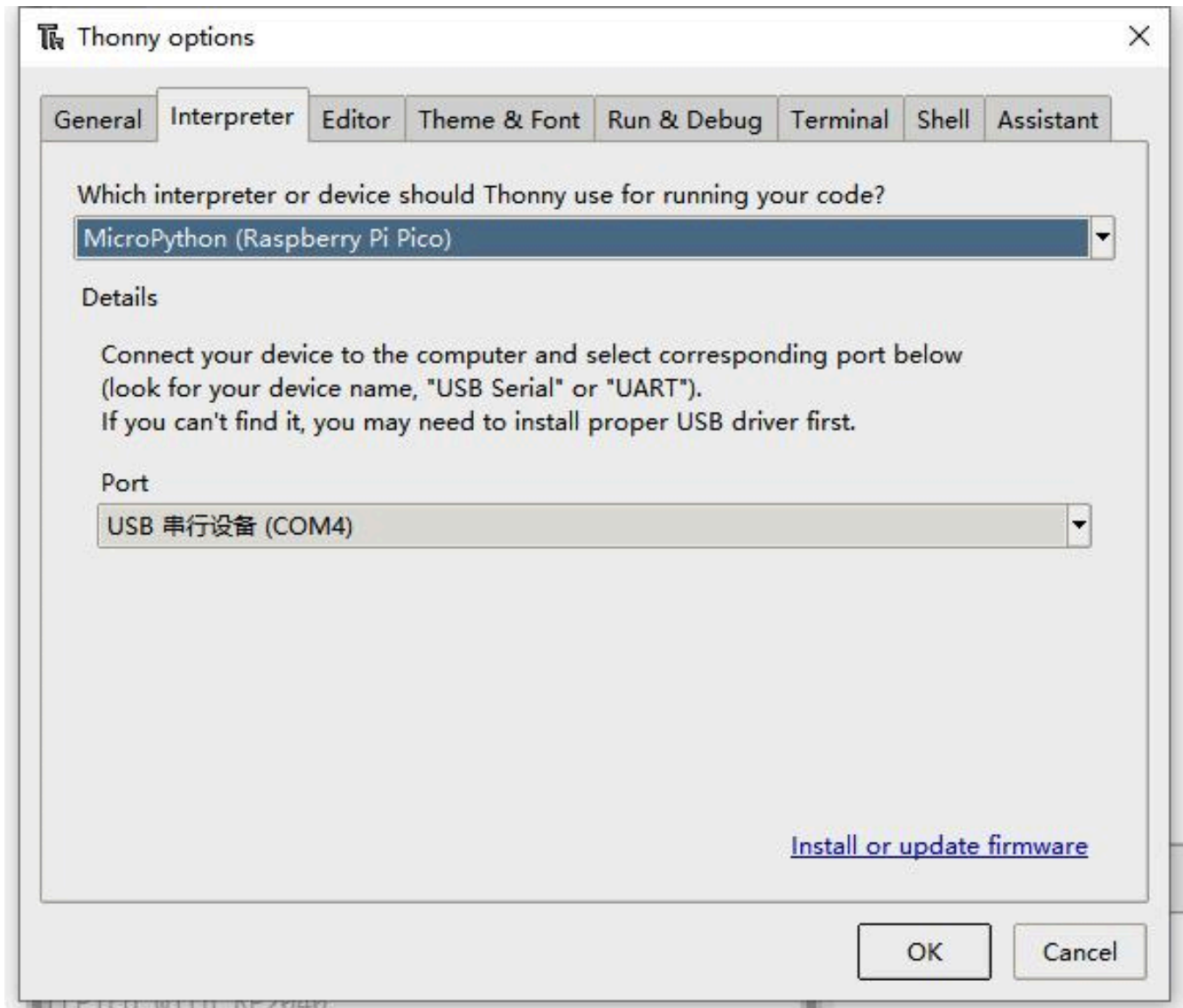
```
#Pico
cp main.uf2 /media/pi/RPI-RP2/
#Pico2
cp main.uf2 /media/pi/RP2350
```

Python Codes

Use in Windows

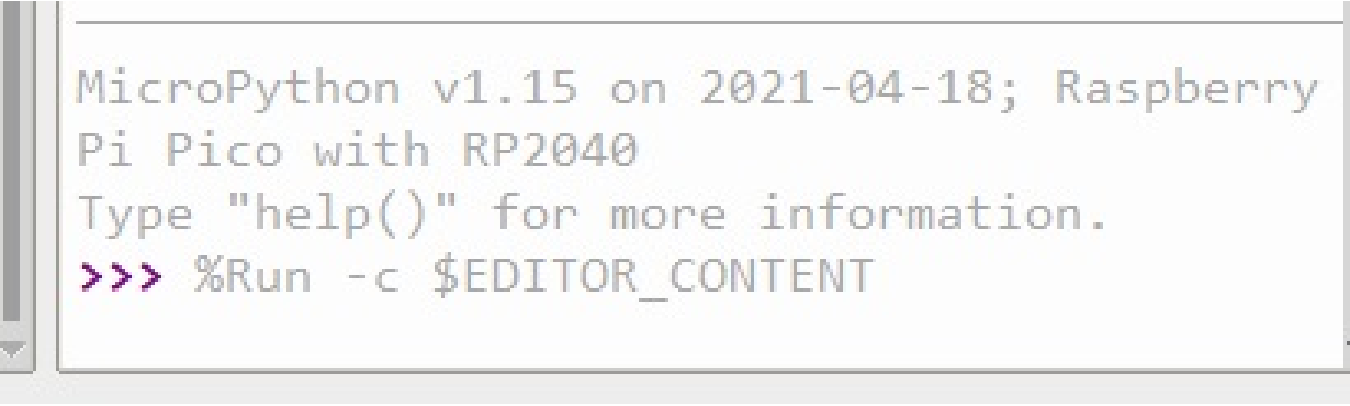
- 1. Press and hold the BOOTSET button on the Pico board, connect the Pico to the USB port of the computer through the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).
- 2. Copy the rp2-pico-20210418-v1.15.uf2 file in the python directory to the recognized removable disk (RPI-RP2).
 - Pico: rp2-pico-20210418-v1.15.uf2 (<https://files.waveshare.com/upload/5/5a/Rp2-pico-20210418-v1.15.7z>)
 - Pico 2: rp2-pcio2-20240809-v1.24.0.uf2 (https://files.waveshare.com/upload/5/5a/RPI_PICO2-20240809-v1.24.0.zip)

- 3. Open Thonny IDE (Note: Use the latest version of Thonny, otherwise there is no Pico support package, the latest version under Windows is v3.3.3).
- 4. Click Tools->Settings->Interpreter, and select Pico and the corresponding port as shown in the figure.



(/wiki/File:Pico-lcd-0.96-img-config.png)

- 5. File -> Open -> the corresponding .py file, click to run, as shown in the following figure:



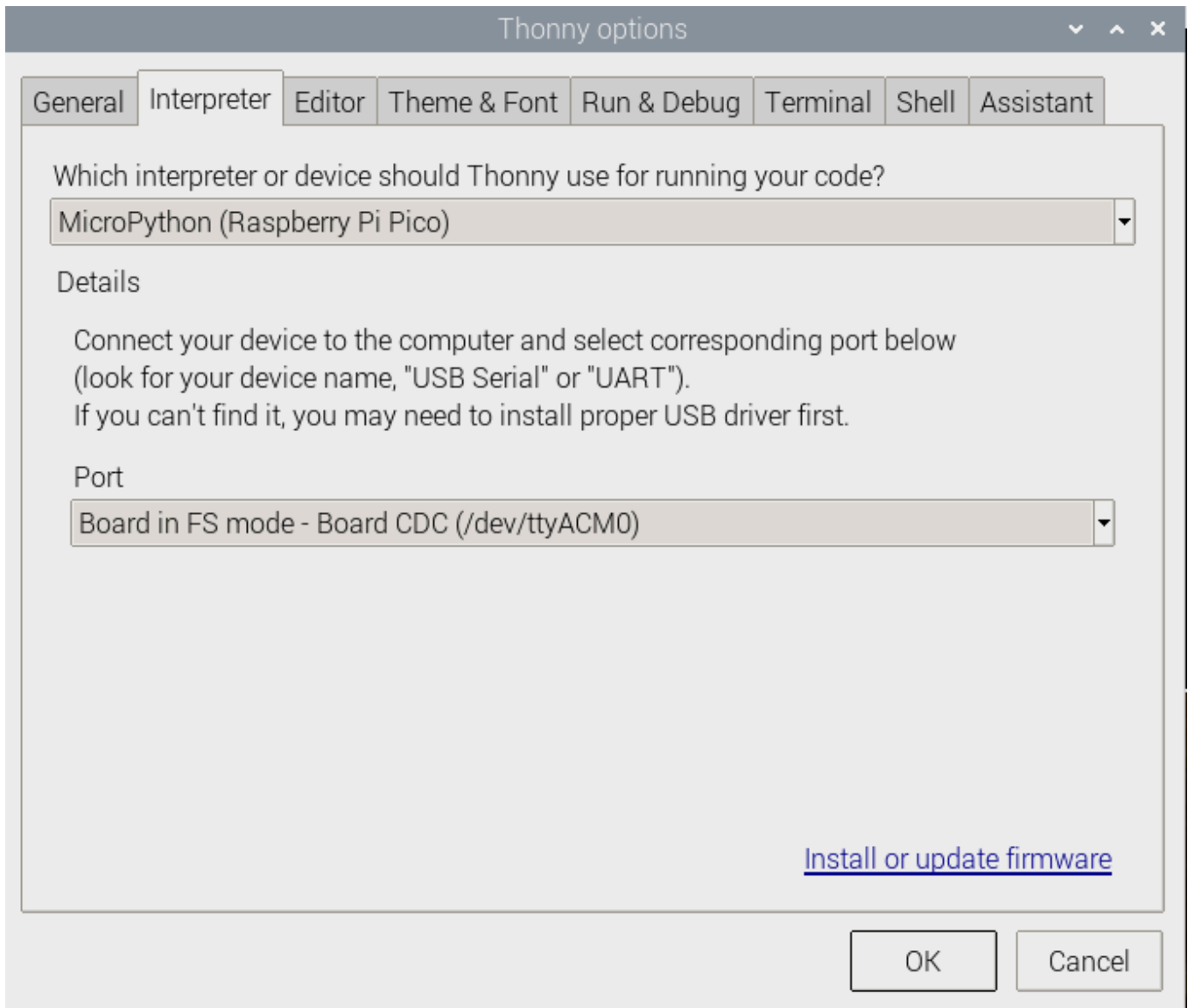
```
MicroPython v1.15 on 2021-04-18; Raspberry
Pi Pico with RP2040
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
```

(/wiki/File:Pico-Thonny-v1.15.jpg)

This demo provides a simple program...

Run in Raspberry Pi

- 1. The process of flashing the firmware is the same as on Windows, and you have the option of copying the .uf2 format file into the Pico/Pico2 on your PC or Raspberry Pi.
- 2. Open the Thonny IDE on the Raspberry Pi (click on the Raspberry logo -> Programming -> Thonny Python IDE) and you can view the version information at Help -> About Thonny.
- 3. Open the Thonny IDE in Raspberry Pi, update it if it doesn't support Pico
- 4. Configure the port by choosing MicroPython(Raspberry Pi and ttyACM0 port) in Tools -> Options... -> Interpreter



(/wiki/File:Pico-lcd-0.96-img-config2.png)

If your Thonny doesn't support Pico, you can update it with the following command:

```
sudo apt upgrade thonny
```

- Choose File -> Open...-> python/ and select the corresponding .py file to run the codes.

Codes Analysis (C)

Note: The libraries used for the RGB LED are similar to the codes of LCD.

Application functions

We provide basic GUI functions for testing, like draw point, line, string, and so on. The GUI function can be found in directory:...\c\lib\GUI\GUI_Paint.c(.h)

 GUI_Paint.c	2021/2/1 11:18	C 文件	32 KB
 GUI_Paint.h	2021/2/1 11:17	H 文件	6 KB

(/wiki/File:LCD_PICO_GUI_1.png)

The fonts used can be found in the directory: RaspberryPi\c\lib\Fonts

名称	修改日期	类型	大小
 font8.c	2020/5/20 11:58	C 文件	18 KB
 font12.c	2020/5/20 11:58	C 文件	27 KB
 font12CN.c	2020/6/5 18:57	C 文件	6 KB
 font16.c	2020/5/20 11:58	C 文件	49 KB
 font20.c	2020/5/20 11:58	C 文件	65 KB
 font24.c	2020/5/20 11:58	C 文件	97 KB
 font24CN.c	2020/6/5 19:01	C 文件	28 KB
 fonts.h	2020/5/20 11:58	H 文件	4 KB

(/wiki/File:LCD_rpi_Font.png)

- Create a new image, you can set the image name, width, height, rotate angle, and color.

```
void Paint_NewImage(UWORD *image, UWORD Width, UWORD Height, UWORD Rotate, UWORD Color, UWORD Depth)
```

Parameter:

```
image : Name of the image buffer, this is a pointer;
Width : Width of the image;
Height: Height of the image;
Rotate: Rotate the angle of the Image;
Color : The initial color of the image;
Depth : Depth of the color
```

- Select image buffer: You can create multiple image buffers at the same time select a certain one and draw by this function.

```
void Paint_SelectImage(UBYTE *image)
```

Parameter:

```
image: The name of the image buffer, this is a pointer;
```

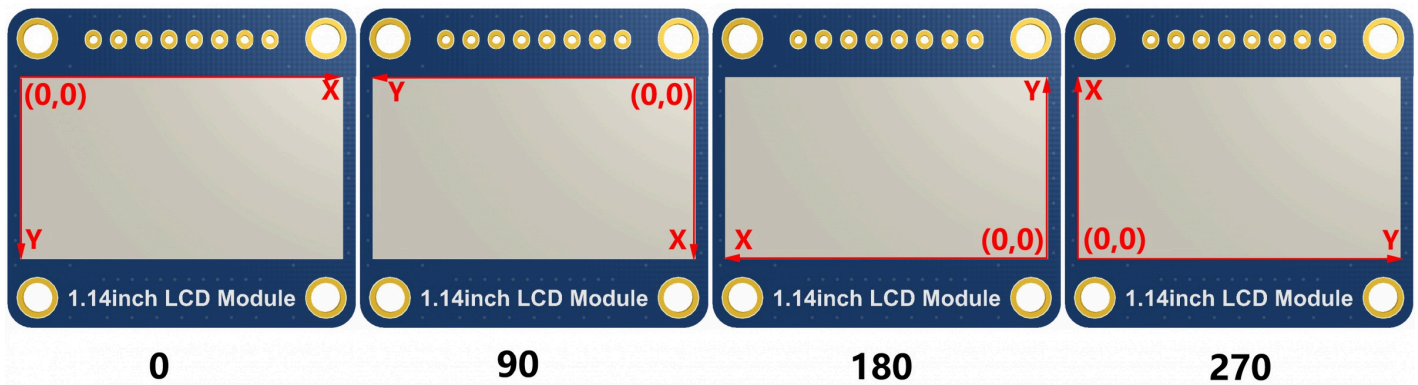
- Rotate image: You need to set the rotation angle of the image, this function should be used after Paint_SelectImage(). The angle can be 0, 90, 180, 270

```
void Paint_SetRotate(UWORD Rotate)
```

Parameter:

```
Rotate: Rotate the angle of the image, the parameter can be ROTATE_0, ROTATE_90, ROTATE_180, ROTATE_270.
```

Note After rotating, the place of the first pixel is different as below



(/wiki/File:Pico_LCD_Rotate.jpg)

- Image mirror: This function is used to set the image mirror.

```
void Paint_SetMirroring(UBYTE mirror)
```

Parameter:

mirror: Mirror type if the image, the parameter can be MIRROR_NONE MIRROR_HORIZONTAL MIRROR_VERTICAL MIRROR_ORIGIN.

- Set the position and color of pixels: This is the basic function of GUI, it is used to set the position and color of a pixel in the buffer.

```
void Paint_SetPixel(UWORD Xpoint, UWORD Ypoint, UWORD Color)
```

Parameter:

Xpoint: The X-axis position of the point in the image buffer

Ypoint: The Y-axis position of the point in the image buffer

Color : The color of the point

- Color of the image: To set the color of the image, this function always be used to clear the display.

```
void Paint_Clear(UWORD Color)
```

Parameter:

Color: The color of the image

- Color of the windows: This function is used to set the color of windows, it is always used for updating partial areas like displaying a clock.

```
void Paint_ClearWindows(UWORD Xstart, UWORD Ystart, UWORD Xend, UWORD Yend, UWORD Color)
```

Parameter:

Xstart: X-axis position of the start point.

Ystart: Y-axis position of the start point.

Xend: X-axis position of the endpoint.

Yend: Y-axis position of the endpoint

Color: Color of the windows.

- Draw point: Draw a point at the position Xpoint, Ypoint of image buffer, you can configure the color, size, and style.

```
void Paint_DrawPoint(UWORD Xpoint, UWORD Ypoint, UWORD Color, DOT_PIXEL Dot_Pixel,
DOT_STYLE Dot_Style)
```

Parameter:

Xpoint: X-axis position of the point.

Ypoint: Y-axis position of the point

Color: Color of the point

Dot_Pixel: Size of the point, 8 sizes are available.

```
typedef enum {
    DOT_PIXEL_1X1  = 1,    // 1 x 1
    DOT_PIXEL_2X2  ,        // 2 X 2
    DOT_PIXEL_3X3  ,        // 3 X 3
    DOT_PIXEL_4X4  ,        // 4 X 4
    DOT_PIXEL_5X5  ,        // 5 X 5
    DOT_PIXEL_6X6  ,        // 6 X 6
    DOT_PIXEL_7X7  ,        // 7 X 7
    DOT_PIXEL_8X8  ,        // 8 X 8
} DOT_PIXEL;
```

Dot_Style: Style of the point, it defines the extended mode of the point.

```
typedef enum {
    DOT_FILL_AROUND  = 1,
    DOT_FILL_RIGHTUP,
} DOT_STYLE;
```

- Draw line: Draw line from (Xstart, Ystart) to (Xend, Yend) in the image buffer, you can configure the color, width, and style.

```
void Paint_DrawLine(UWORD Xstart, UWORD Ystart, UWORD Xend, UWORD Yend, UWORD Color,
LINE_STYLE Line_Style , LINE_STYLE Line_Style)
```

Parameter:

Xstart: Xstart of the line

Ystart: Ystart of the line

Xend: Xend of the line

Yend: Yend of the line

Color: Color of the line

Line_width: Width of the line, 8 sizes are available.

```
typedef enum {
    DOT_PIXEL_1X1  = 1,    // 1 x 1
    DOT_PIXEL_2X2  ,        // 2 X 2
    DOT_PIXEL_3X3  ,        // 3 X 3
    DOT_PIXEL_4X4  ,        // 4 X 4
    DOT_PIXEL_5X5  ,        // 5 X 5
    DOT_PIXEL_6X6  ,        // 6 X 6
    DOT_PIXEL_7X7  ,        // 7 X 7
    DOT_PIXEL_8X8  ,        // 8 X 8
} DOT_PIXEL;
```

Line_Style: Style of the line, Solid or Dotted.

```
typedef enum {
    LINE_STYLE_SOLID = 0,
    LINE_STYLE_DOTTED,
} LINE_STYLE;
```

- Draw rectangle: Draw a rectangle from (Xstart, Ystart) to (Xend, Yend), you can configure the color, width, and style.

```
void Paint_DrawRectangle(UWORD Xstart, UWORD Ystart, UWORD Xend, UWORD Yend, UWORD
Color, DOT_PIXEL Line_width, DRAW_FILL Draw_Fill)
```

Parameter:

Xstart: Xstart of the rectangle.

Ystart: Ystart of the rectangle.

Xend: Xend of the rectangle.

Yend: Yend of the rectangle.

Color: Color of the rectangle

Line_width: The width of the edges. 8 sizes are available.

```
typedef enum {
    DOT_PIXEL_1X1  = 1,    // 1 x 1
    DOT_PIXEL_2X2  ,        // 2 X 2
    DOT_PIXEL_3X3  ,        // 3 X 3
    DOT_PIXEL_4X4  ,        // 4 X 4
    DOT_PIXEL_5X5  ,        // 5 X 5
    DOT_PIXEL_6X6  ,        // 6 X 6
    DOT_PIXEL_7X7  ,        // 7 X 7
    DOT_PIXEL_8X8  ,        // 8 X 8
} DOT_PIXEL;
```

Draw_Fill: Style of the rectangle, empty or filled.

```
typedef enum {
    DRAW_FILL_EMPTY = 0,
    DRAW_FILL_FULL,
} DRAW_FILL;
```

- Draw circle: Draw a circle in the image buffer, use (X_Center Y_Center) as center and Radius as radius. You can configure the color, width of the line, and style of the circle.

```
void Paint_DrawCircle(UWORD X_Center, UWORD Y_Center, UWORD Radius, UWORD Color, DOT_PIXEL Line_width, DRAW_FILL Draw_Fill)
```

Parameter:

X_Center: X-axis of center

Y_Center: Y-axis of center

Radius radius of circle

Color: Color of the circle

Line_width: The width of arc, 8 sizes are available.

```
typedef enum {
    DOT_PIXEL_1X1  = 1,    // 1 x 1
    DOT_PIXEL_2X2  ,      // 2 x 2
    DOT_PIXEL_3X3  ,      // 3 x 3
    DOT_PIXEL_4X4  ,      // 4 x 4
    DOT_PIXEL_5X5  ,      // 5 x 5
    DOT_PIXEL_6X6  ,      // 6 x 6
    DOT_PIXEL_7X7  ,      // 7 x 7
    DOT_PIXEL_8X8  ,      // 8 x 8
} DOT_PIXEL;
```

Draw_Fill: Style of the circle: empty or filled.

```
typedef enum {
    DRAW_FILL_EMPTY = 0,
    DRAW_FILL_FULL,
} DRAW_FILL;
```

- Show Ascii character: Show a character in (Xstart, Ystart) position, you can configure the font, foreground and the background.

```
void Paint_DrawChar(UWORD Xstart, UWORD Ystart, const char Ascii_Char, sFONT* Font, UWORD Color_Foreground, UWORD Color_Background)
```

Parameter:

Xstart: Xstart of the character

Ystart: Ystart of the character

Ascii_Char Ascii char

Font: five fonts are available

font8 5*8

font12 7*12

font16 11*16

font20 14*20

font24 17*24

Color_Foreground: foreground color

Color_Background: background color

- Draw string: Draw string at (Xstart Ystart) , you can configure the fonts, foreground and the background

```
void Paint_DrawString_EN(UWORD Xstart, UWORD Ystart, const char * pString, sFONT* Font, UWORD Color_Foreground, UWORD Color_Background)
```

Parameter:

Xstart: Xstart of the string
 Ystart: Ystart of the string
 pString String
 Font: five fonts are available
 font8 5*8
 font12 7*12
 font16 11*16
 font20 14*20
 font24 17*24
 Color_Foreground: foreground color
 Color_Background: background color

- Draw Chinese string: Draw Chinese string at (Xstart Ystart) of image buffer. You can configure fonts (GB2312), foreground and the background.

```
void Paint_DrawString_CN(UWORD Xstart, UWORD Ystart, const char * pString, cFONT* font, UWORD Color_Foreground, UWORD Color_Background)
```

Parameter:

Xstart: Xstart of string
 Ystart: Ystart of string
 pString string
 Font: GB2312 fonts, two fonts are available
 font12CN ascii 11*21 Chinese 16*21
 font24CN ascii 24*41 Chinese 32*41
 Color_Foreground: Foreground color
 Color_Background: Background color

- Draw number: Draw numbers at (Xstart Ystart) of image buffer. You can select font, foreground and the background.

```
void Paint_DrawNum(UWORD Xpoint, UWORD Ypoint, int32_t Number, sFONT* Font, UWORD Color_Foreground, UWORD Color_Background)
```

Parameter:

Xstart: Xstart of numbers
 Ystart: Ystart of numbers
 Number numbers displayed. It support int type and 2147483647 are the maximum supported
 Font: Ascii fonts, five fonts are available:
 font8 5*8
 font12 7*12
 font16 11*16
 font20 14*20
 font24 17*24
 Color_Foreground: Foreground color
 Color_Background: Background color

- Draw float numbers: Draw float number at (Xstart Ystart) of the image buffer, you can configure fonts, foreground, and background.

```
void Paint_DrawFloatNum(UWORD Xpoint, UWORD Ypoint, double Number, UBYTE Decimal_Point, sFONT* Font, UWORD Color_Foreground, UWORD Color_Background);
```

Parameter:

Xstart: Xstart of the number

Ystart: Ystart of the number

Nummber: The float number. Double type.

Decimal_Point: The decimal number

Font: Ascii fonts, five fonts are available.

font8 5*8

font12 7*12

font16 11*16

font20 14*20

font24 17*24

Color_Foreground: Foreground

Color_Background: Background

- Display time: Display time at (Xstart Ystart) of the image buffer, you can configure fonts, foreground, and background.

```
void Paint_DrawTime(UWORD Xstart, UWORD Ystart, PAINT_TIME *pTime, sFONT* Font, UWORD Color_Background, UWORD Color_Foreground)
```

Parameter:

Xstart: Xstart of time

Ystart: Ystart of time

pTime Structure of time

Font: Ascii font, five fonts are available

font8 5*8

font12 7*12

font16 11*16

font20 14*20

font24 17*24

Color_Foreground: Foreground

Color_Background: Background

Resource

Document

- Schematic (https://files.waveshare.com/upload/c/c3/Pico_RGB_LED_SchDoc.pdf)

Examples

- Demo codes (https://files.waveshare.com/upload/5/5a/Pico-RGB-LED_code.7z)

Development Software

- Zimo221.7z (<https://files.waveshare.com/upload/c/c6/Zimo221.7z>)
- Image2Lcd.7z (<https://files.waveshare.com/upload/3/36/Image2Lcd.7z>)
- Font Library Tutorial (https://www.waveshare.com/wiki/Ink_Screen_Font_Library_Tutorial)
- Image Extraction Tutorial (https://www.waveshare.com/wiki/Image_Extraction)
- Thonny Python IDE (Windows V3.3.3) (<https://files.waveshare.com/upload/7/73/Thonny-3.3.3.zip>)

Pico Getting Started

Firmware Download

- MicroPython Firmware Download [\[Expand\]](#)
- C_Blink Firmware Download [\[Expand\]](#)

Introduction

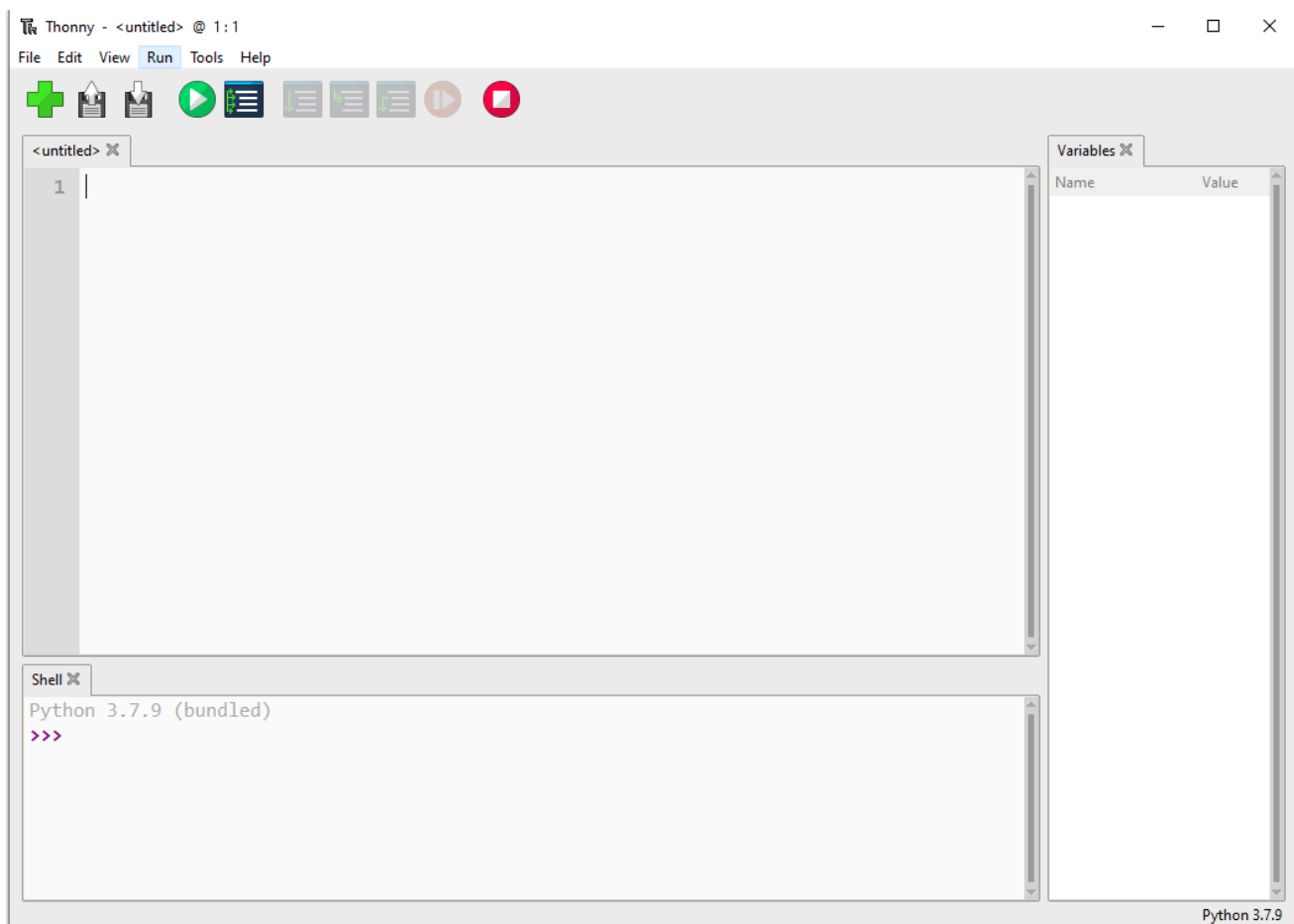
Raspberry Pi Pico Basics (https://www.waveshare.com/wiki/Pico_Basic_Introduction)

MicroPython Series

Install Thonny IDE

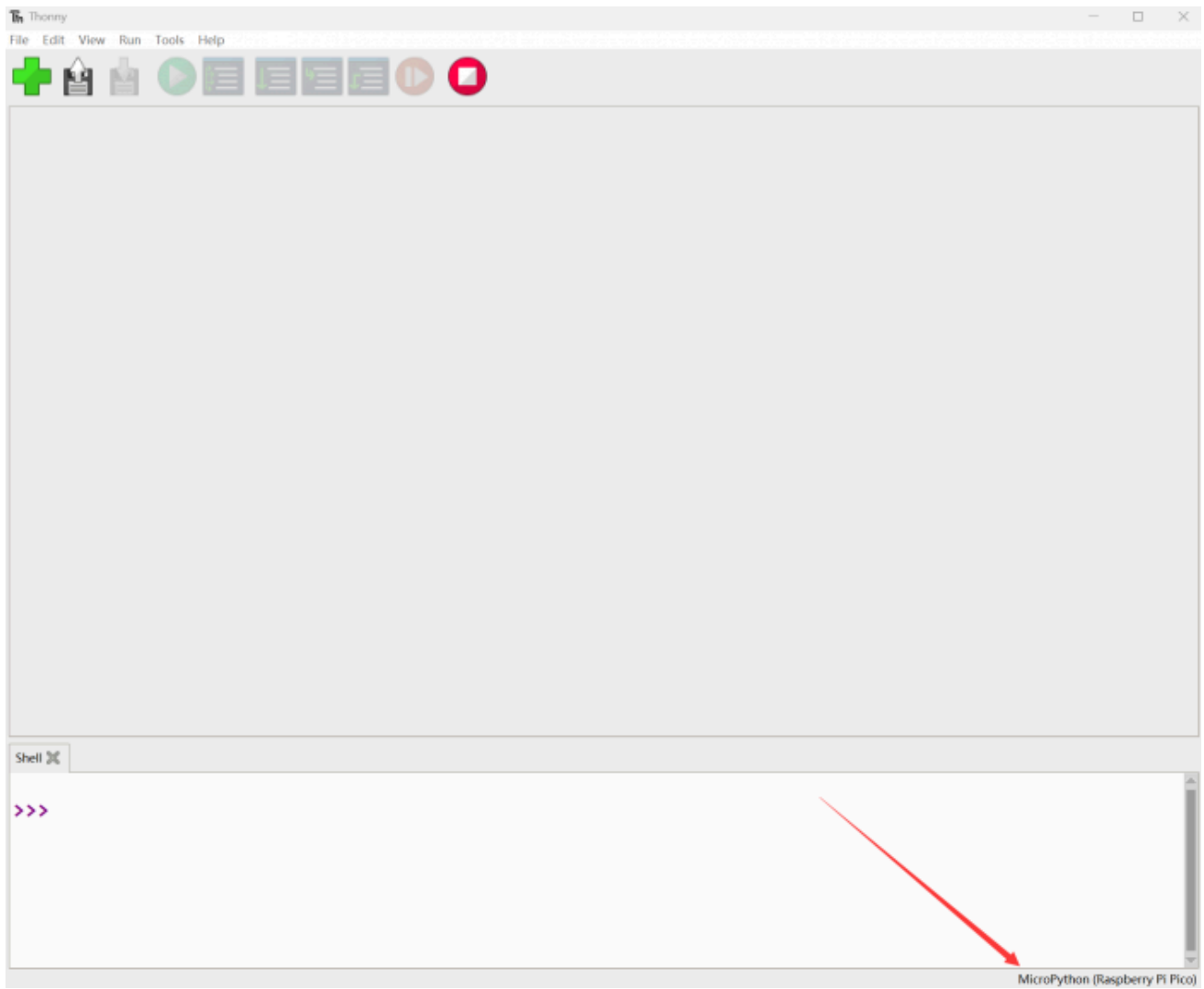
In order to facilitate the development of Pico/Pico2 boards using MicroPython on a computer, it is recommended to download the Thonny IDE

- Download Thonny IDE and follow the steps to install, the installation packages are all Windows versions, please refer to Thonny's official website for other versions
 - Thonny IDE official download link (<https://github.com/thonny/thonny/releases/download/v3.3.3/thonny-3.3.3.exe>)
 - Thonny IDE download link (<https://files.waveshare.com/wiki/common/Thonny-3.3.3.zip>)
 - Thonny official website (<https://thonny.org/>)
- After installation, the language and motherboard environment need to be configured for the first use. Since we are using Pico/Pico2, pay attention to selecting the Raspberry Pi option for the motherboard environment

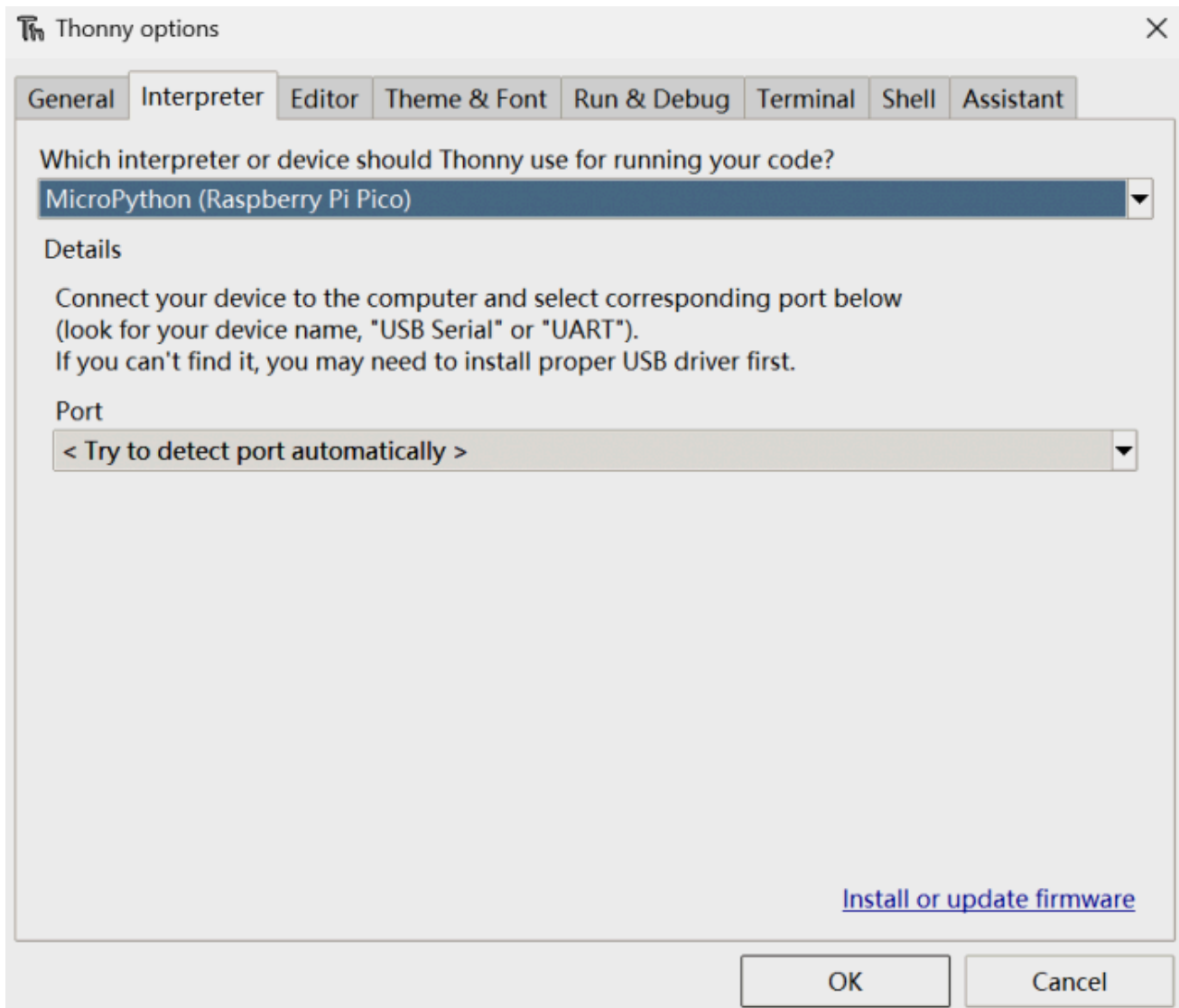


(/wiki/File:Pico-R3-Tonny1.png)

- Configure MicroPython environment and choose Pico/Pico2 port
 - Connect Pico/Pico2 to your computer first, and in the lower right corner of Thonny left-click on the configuration environment option --> select Configure interpreter
 - In the pop-up window, select MicroPython (Raspberry Pi Pico), and choose the corresponding port



(/wiki/File:700px-Raspberry-Pi-Pico-Basic-Kit-M-2.png)



(/wiki/File:700px-Raspberry-Pi-Pico-Basic-Kit-M-3.png)

Flash Firmware

- Click OK to return to the Thonny main interface, download the corresponding firmware library and flash it to the device, and then click the Stop button to display the current environment in the Shell window
- **Note: Flashing the Pico2 firmware provided by Micropython may cause the device to be unrecognized, please use the firmware below or in the package**
 - Pico firmware library (https://files.waveshare.com/wiki/Raspberry-Pi-Pico-2/RPI_PICO-20250415-v1.25.0.zip)
 - Pico2 firmware library (<https://files.waveshare.com/wiki/common/WAVESHARE-RP2350-20250711-v1.26.0.zip>)
 - Pico2 16MB firmware library (<https://files.waveshare.com/wiki/common/WAVESHARE-RP2350-20250807-v1.26.0-16MB.zip>)
 - RP2350B firmware library (https://files.waveshare.com/wiki/common/WAVESHARE_RP2350B.zip)
- How to download the firmware library for Pico/Pico2 in windows: After holding down the BOOT button and connecting to the computer, release the BOOT button, a removable disk will appear on the computer, copy the firmware library into it

- How to download the firmware library for RP2040/RP2350 in windows: After connecting to the computer, press the BOOT key and the RESET key at the same time, release the RESET key first and then release the BOOT key, a removable disk will appear on the computer, copy the firmware library into it (you can also use the Pico/Pico2 method)



(/wiki/File:Raspberry-Pi-Pico2-Python.png)

MicroPython Series

MicroPython machine.Pin class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91Machine.Pin_Functions)

MicroPython machine.PWM class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.PWM_Function)

MicroPython machine.ADC class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.ADC_Function)

MicroPython machine.UART class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.UART_Function)

MicroPython machine.I2C class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.I2C_Function)

MicroPython machine.SPI class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.SPI_Function)

icroPython%E3%80%91machine.SPI_Function)

MicroPython rp2.StateMachine class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91PIO_Function)

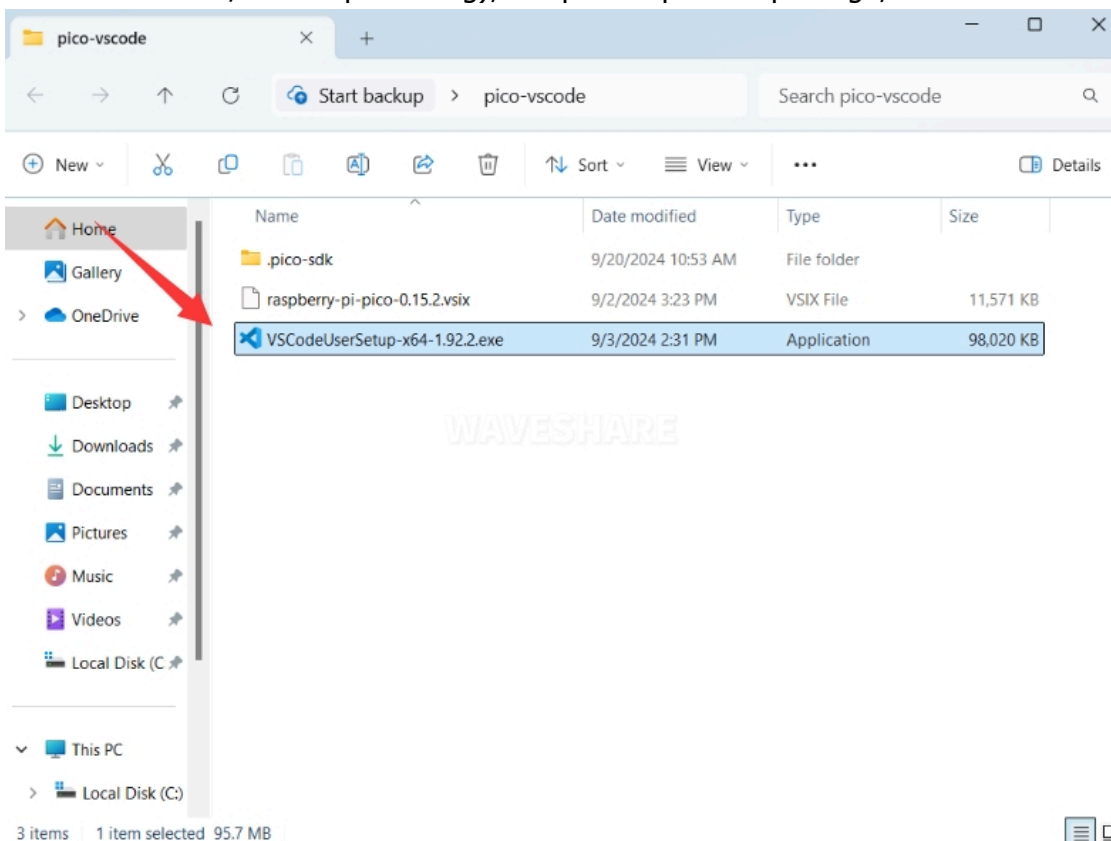
C/C++ Series

For C/C++, it is recommended to use Pico VS Code for development. This is a Microsoft Visual Studio Code extension designed to make it easier for you to create, develop, and debug projects for the Raspberry Pi Pico series development boards. No matter if you are a beginner or an experienced professional, this tool can assist you in developing Pico with confidence and ease. Here's how to install and use the extension.

- Official website tutorial: <https://www.raspberrypi.com/news/pico-vscode-extension/> (<https://www.raspberrypi.com/news/pico-vscode-extension/>)
- This tutorial is suitable for Raspberry Pi Pico, Pico2 and the RP2040 and RP2350 series development boards developed by Waveshare
- The development environment defaults to Windows11. For other environments, please refer to the official tutorial for installation

Install VSCode

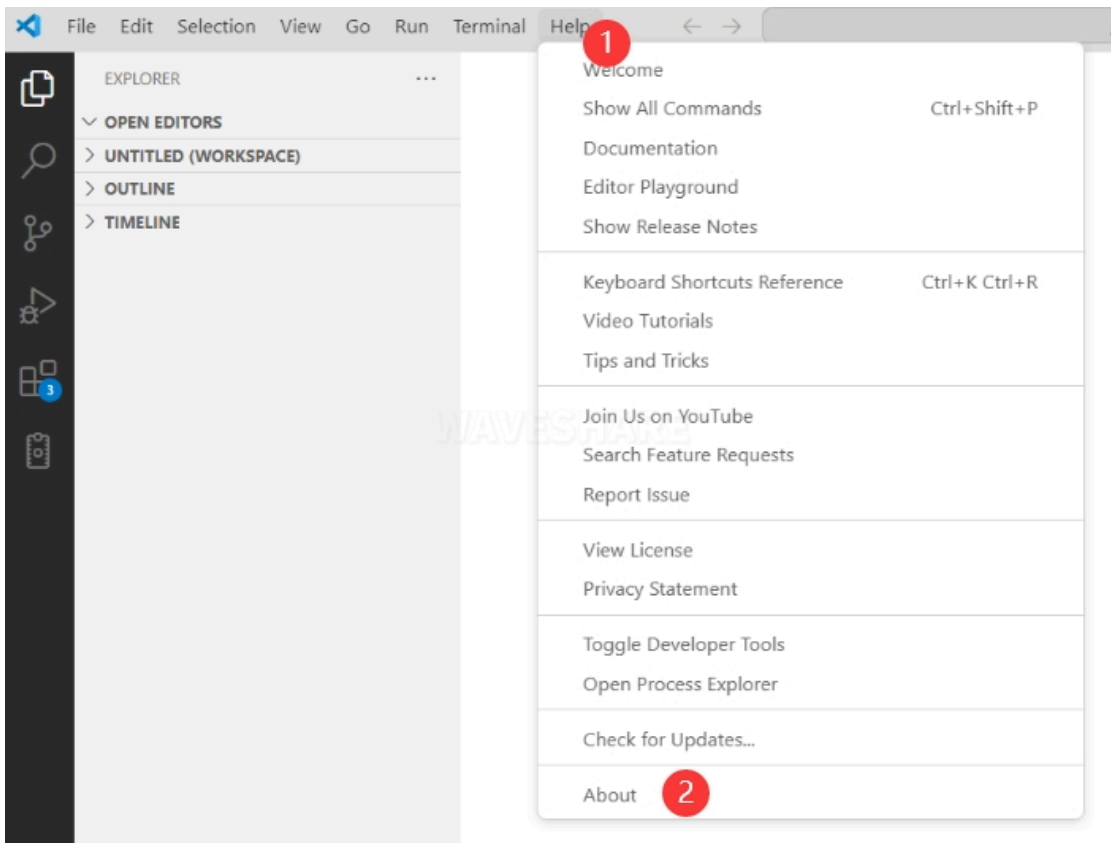
1. First, click to download pico-vscode package (<https://drive.google.com/file/d/18-KDNrQII0KuTMdS6W5ibUGaGm3FbVJ/view?usp=sharing>), unzip and open the package, double-click to install VSCode



(/wiki/File:Pico-

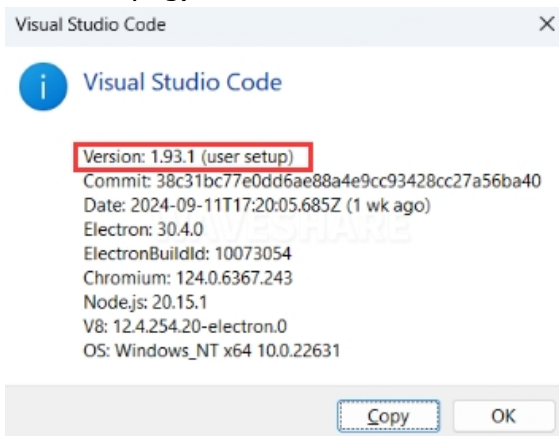
vscode-1.png)

Note: If vscode is installed, check if the version is v1.87.0 or later



(/wiki/File:Pico-

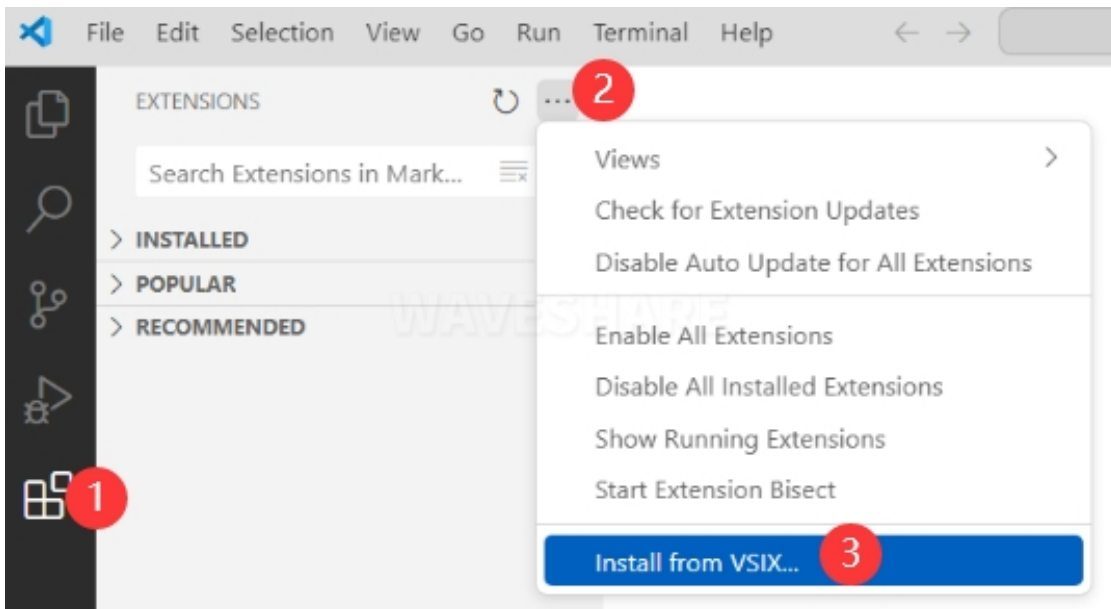
vscode-2.png)



(/wiki/File:Pico-vscode-3.png)

Install Extension

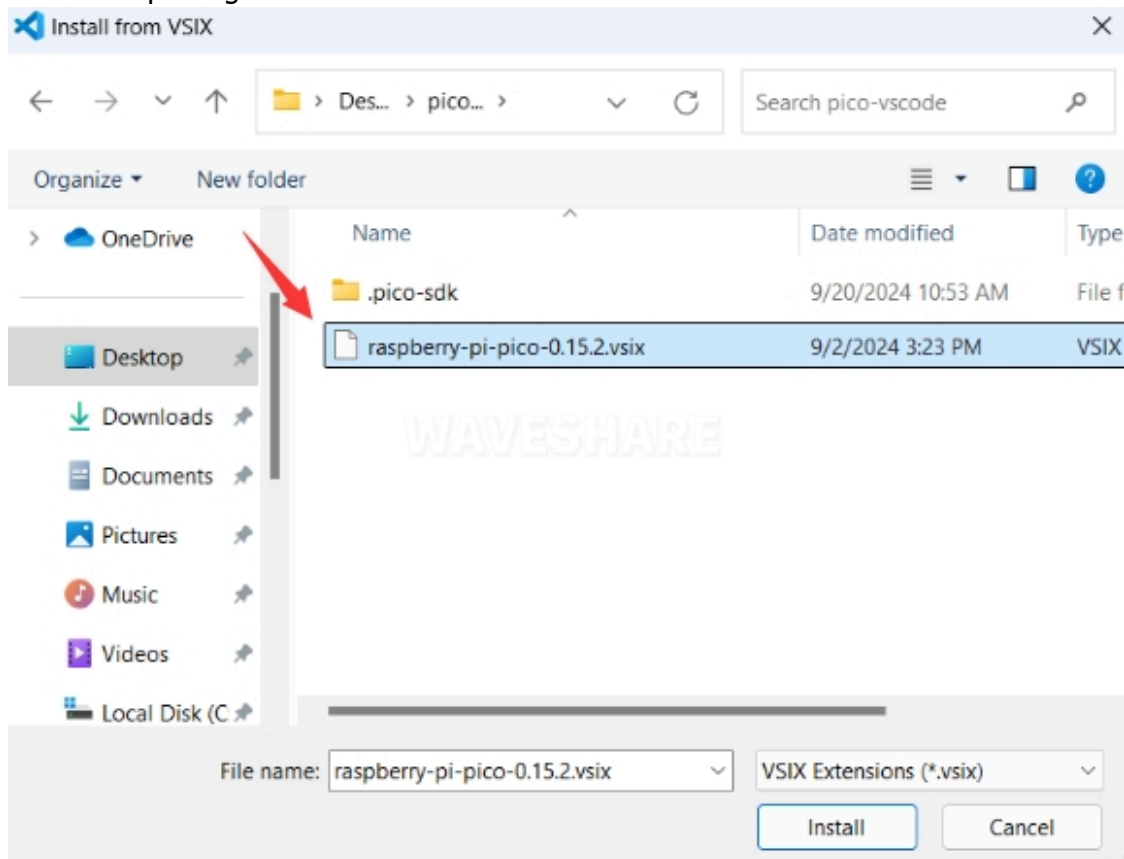
1. Click Extensions and select Install from VSIX



(/wiki/File:Pico-

vscode-4.png)

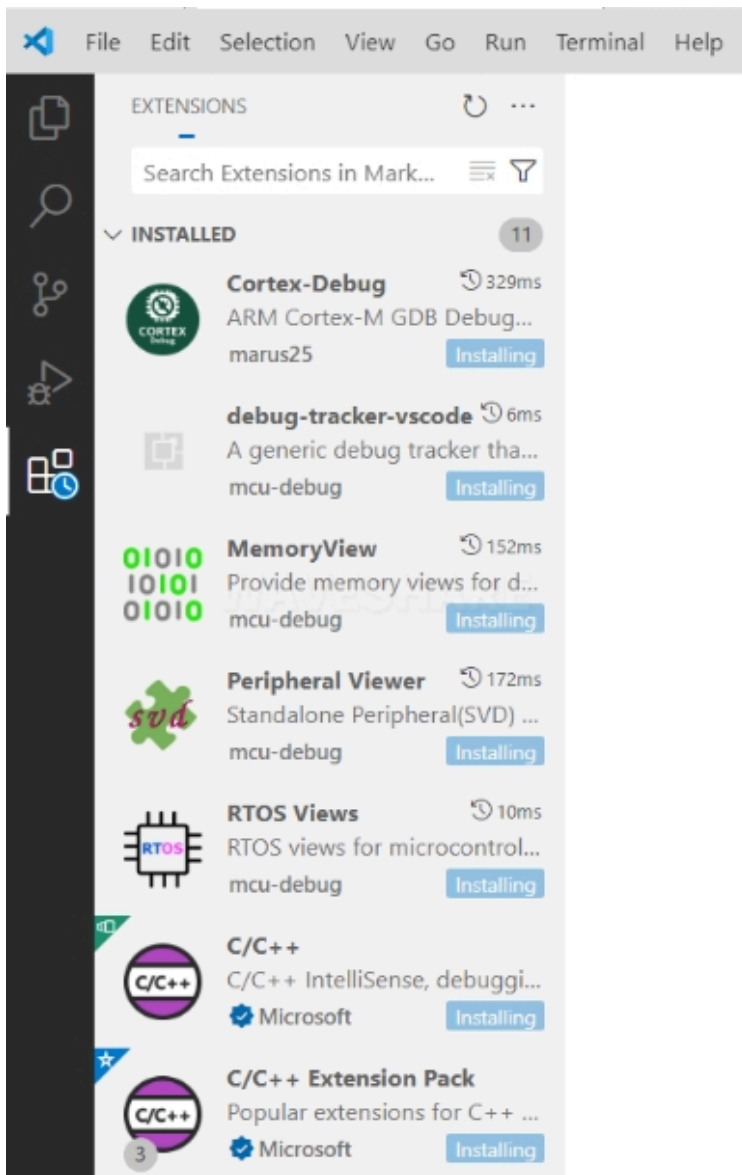
2. Select the package with the vsix suffix and click Install



(/wiki/File:Pico-

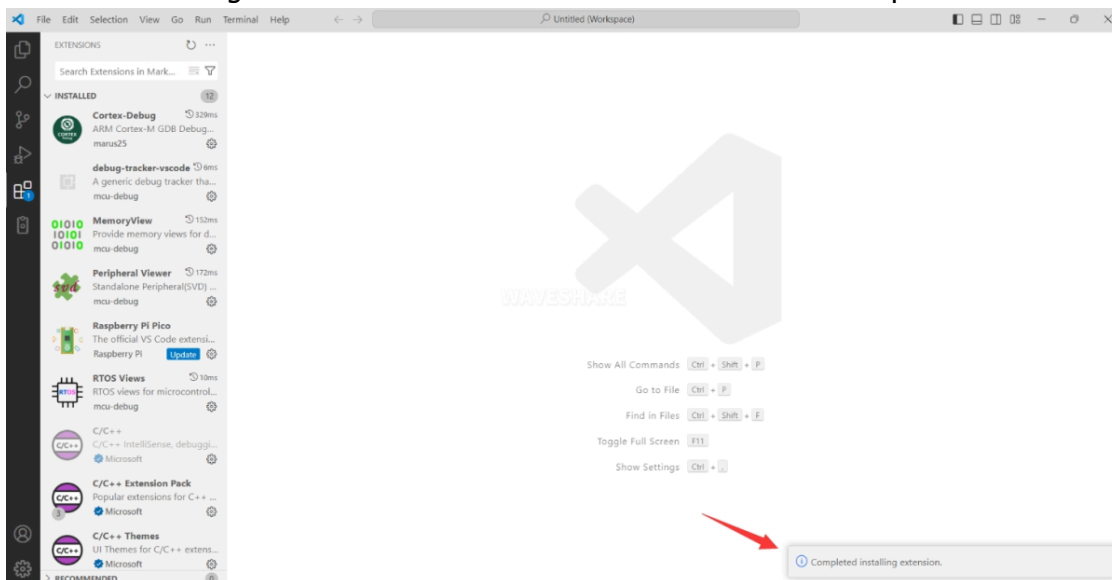
vscode-5.png)

3. Then vscode will automatically install raspberrypi-pico and its dependency extensions, you can click Refresh to check the installation progress



(/wiki/File:Pico-vscode-6.png)

4. The text in the right lower corner shows that the installation is complete. Close VSCode

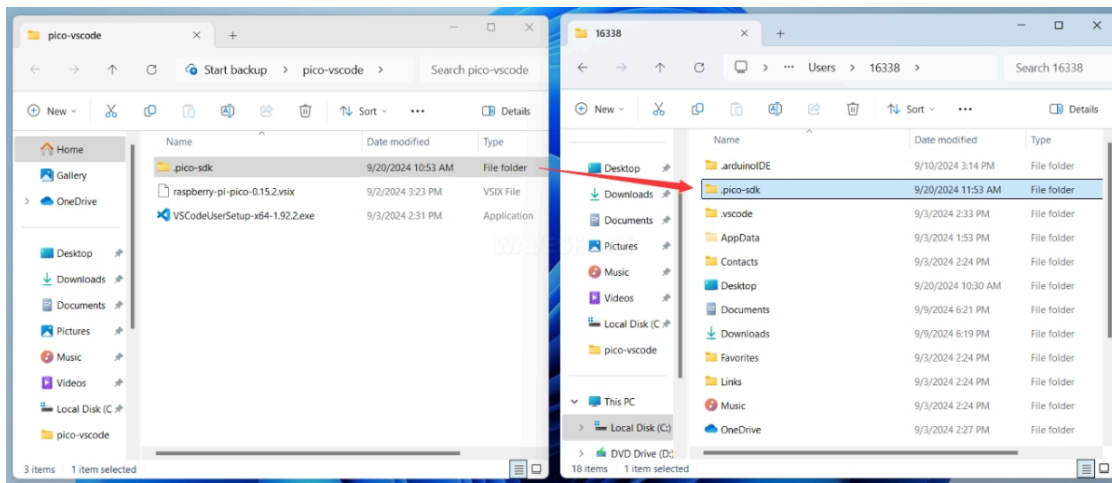


(/wiki/File:Pico-

vscode-7.png)

Configure Extension

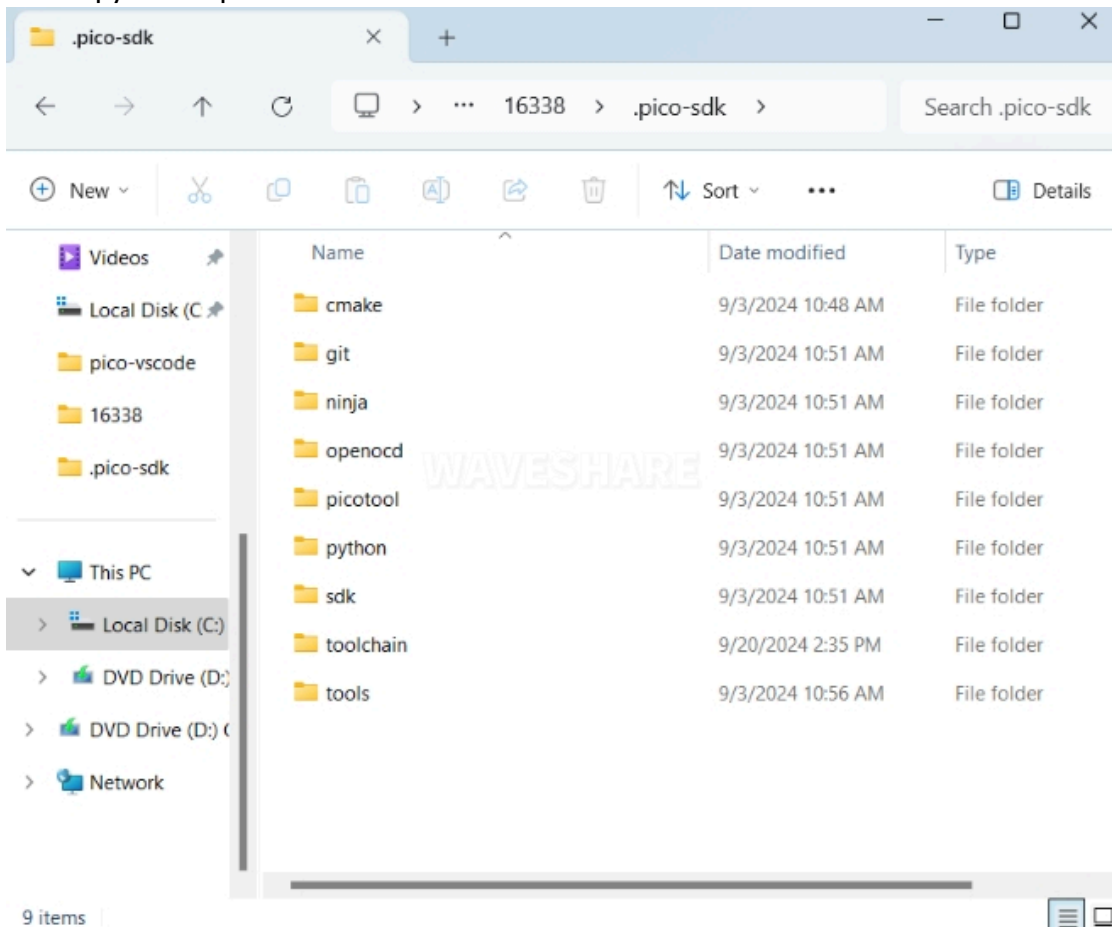
1. Open directory C:\Users\username and copy the entire .pico-sdk to that directory



(/wiki/File:Pico-

vscode-8.png)

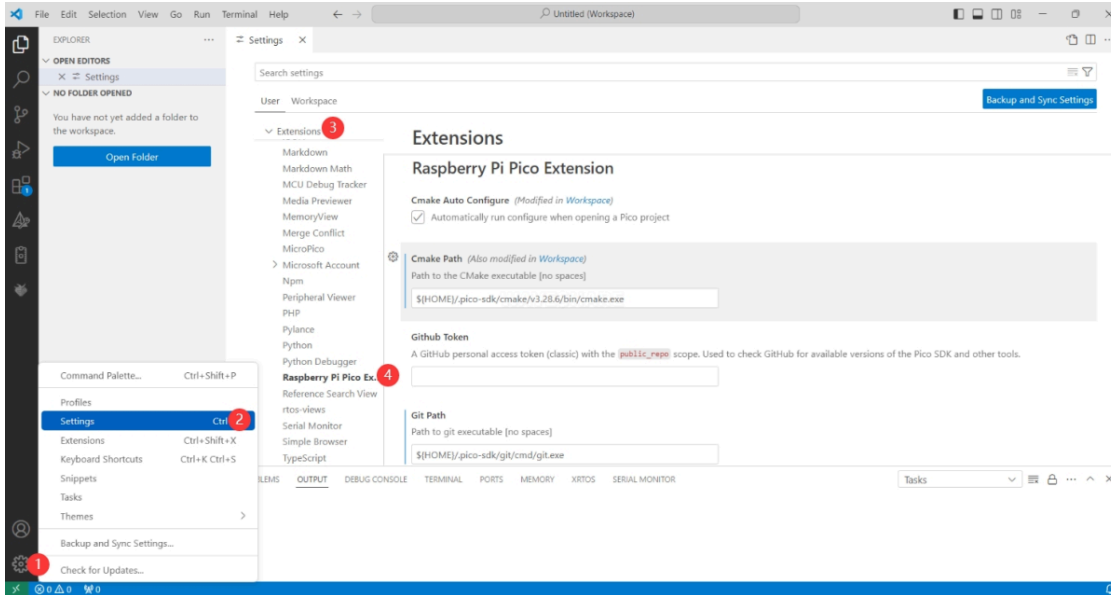
2. The copy is completed



(/wiki/File:Pico-

vscode-9.png)

3. Open vscode and configure the paths for the Raspberry Pi Pico extensions



(/wiki/File:Pico-

vscode-10.png)

The configuration is as follows:

```
Cmake Path:
${HOME}/.pico-sdk/cmake/v3.28.6/bin/cmake.exe

Git Path:
${HOME}/.pico-sdk/git/cmd/git.exe

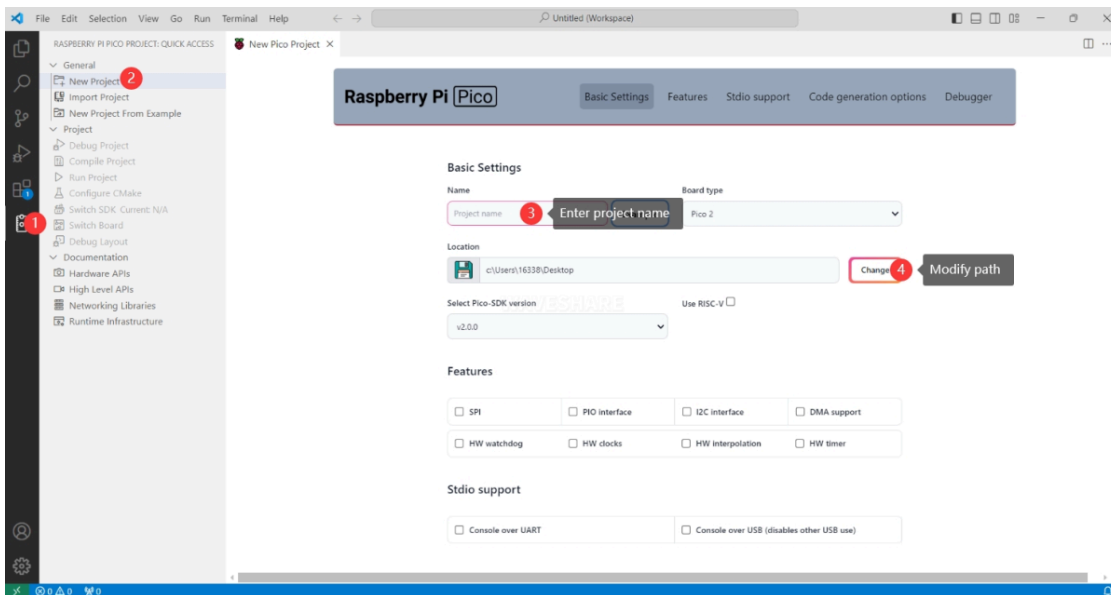
Ninja Path:
${HOME}/.pico-sdk/ninja/v1.12.1/ninja.exe

Python3 Path:
${HOME}/.pico-sdk/python/3.12.1/python.exe
```

New Project

1. The configuration is complete, create a new project, enter the project name, select the path, and click Create to create the project

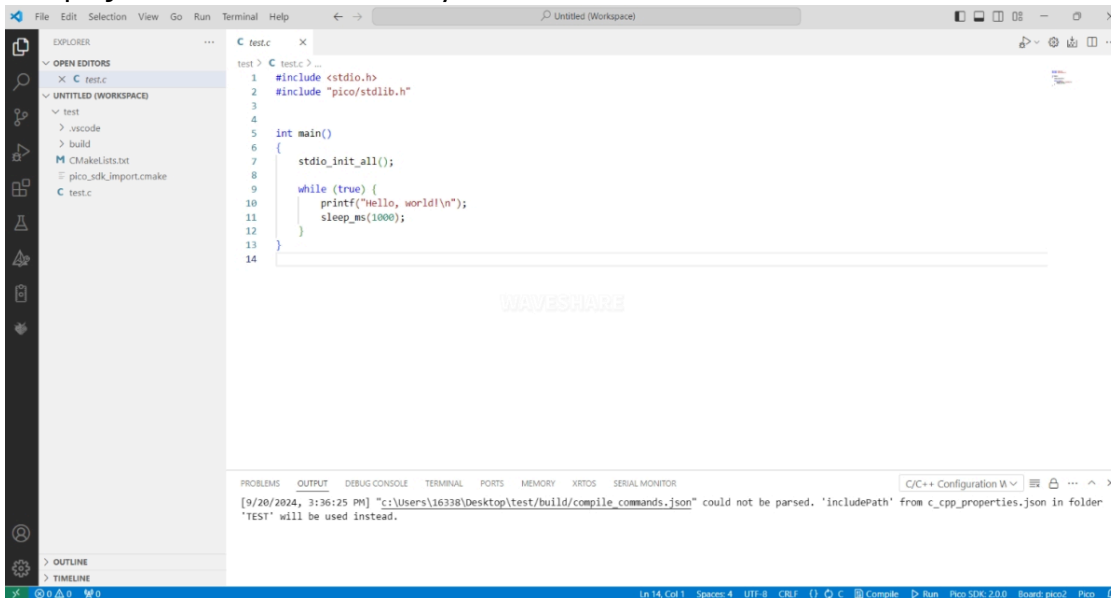
To test the official example, you can click on the Example next to the project name to select



(/wiki/File:Pico-

vscode-11.png)

2. The project is created successfully

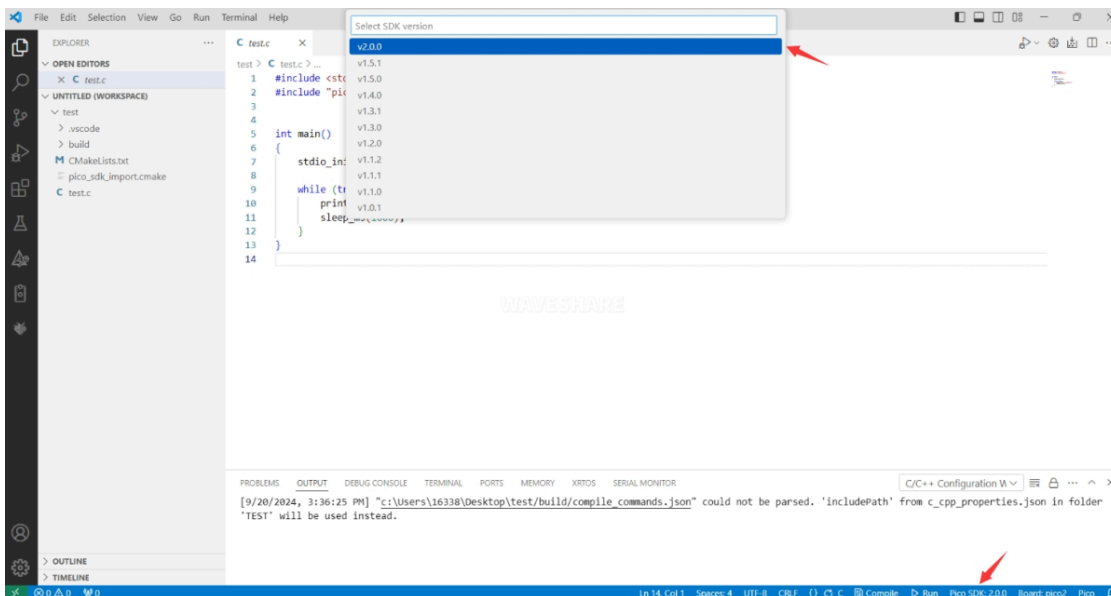


(/wiki/File:Pico-

vscode-12.png)

Compile Project

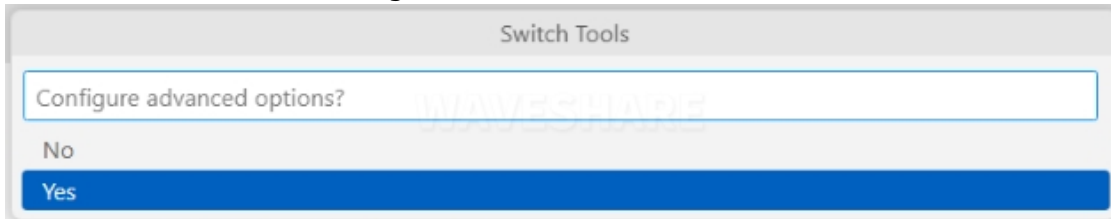
1. Select the SDK version



(/wiki/File:Pico-

vscode-13.png)

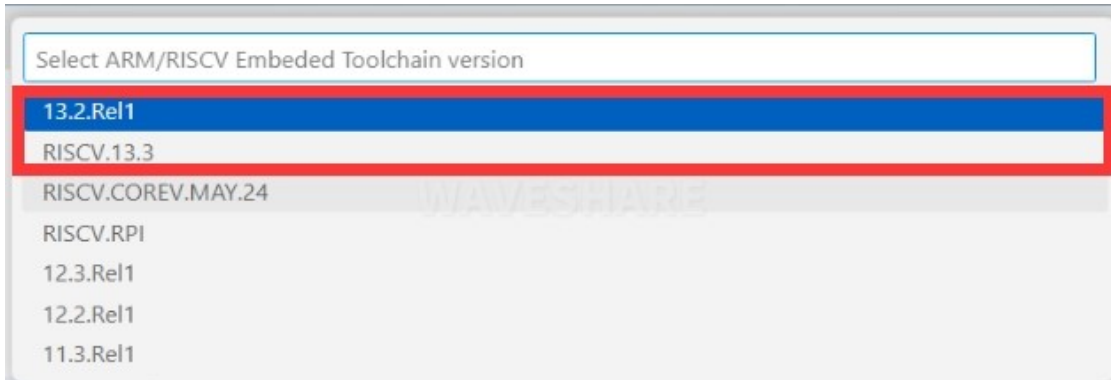
2. Select Yes for advanced configuration



(/wiki/File:Pico-

vscode-14.png)

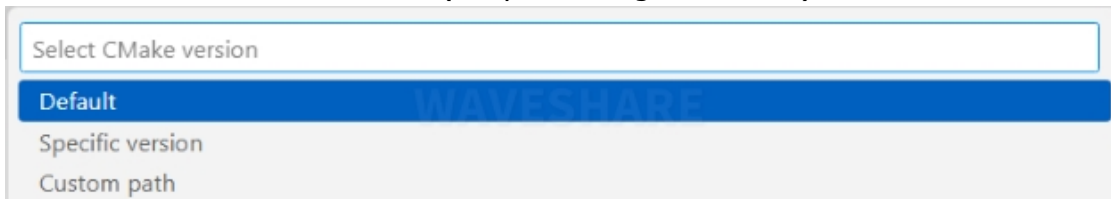
3. Choose the toolchain, 13.2.Rel1 is applicable for ARM cores, RISC.V.13.3 is applicable for RISC.V cores. You can select either based on your requirements



(/wiki/File:Pico-

vscode-15.png)

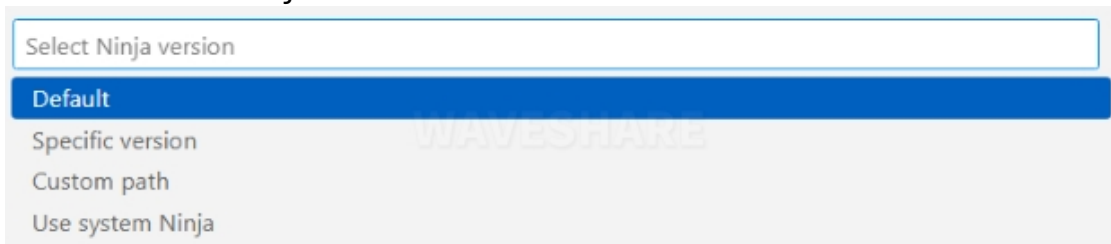
4. Select Default for CMake version (the path configured earlier)



(/wiki/File:Pico-

vscode-16.png)

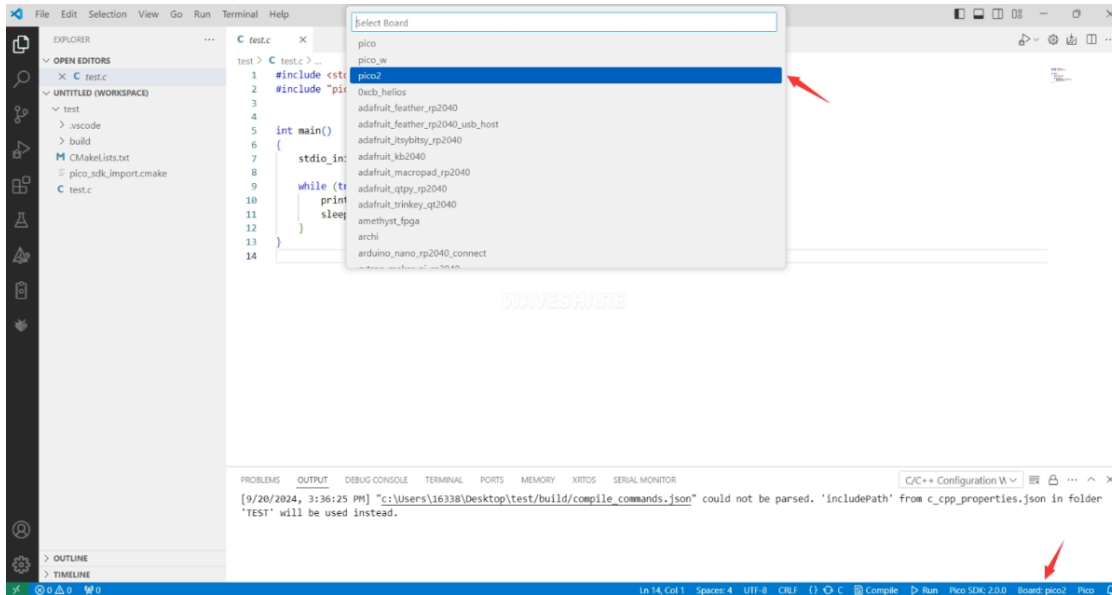
5. Select Default for Ninja version



(/wiki/File:Pico-

vscode-17.png)

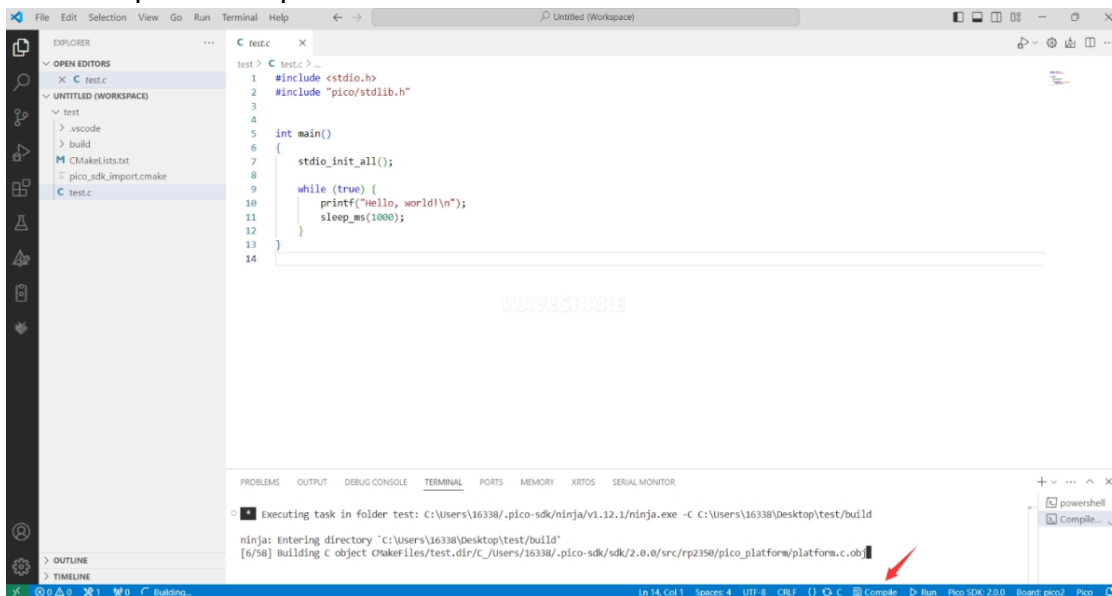
6. Select the development board



(/wiki/File:Pico-

vscode-18.png)

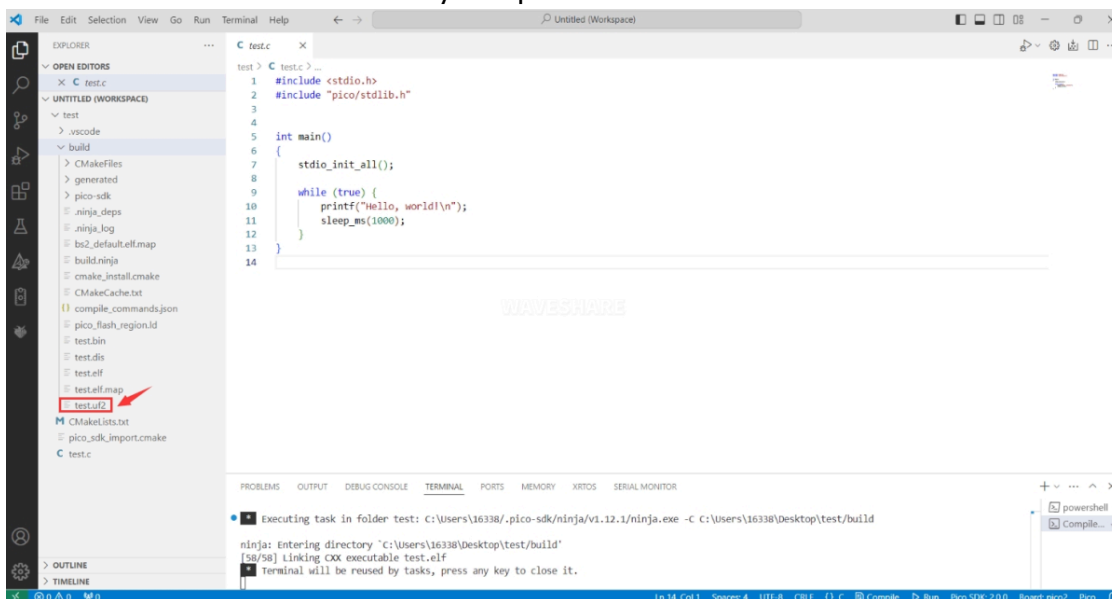
7. Click Compile to compile



(/wiki/File:Pico-

vscode-19.png)

8. The .uf2 format file is successfully compiled



(/wiki/File:Pico-

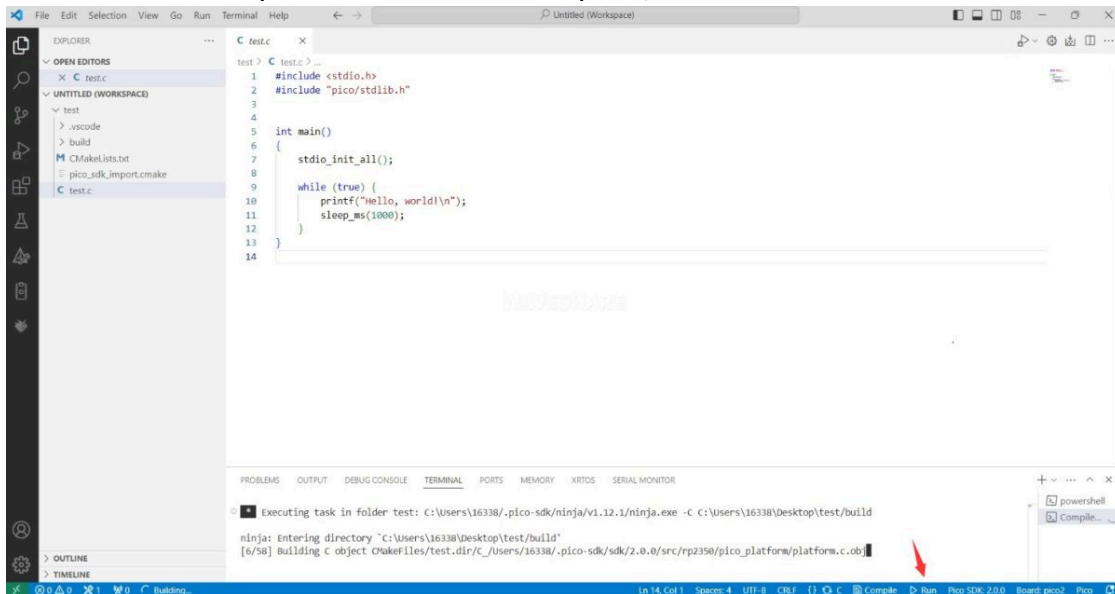
vscode-20.png)

Flash Firmware

Here are two methods for flashing firmware

1. Flash firmware using the pico-vscode plugin

Connect the development board to the computer, click Run to flash the firmware directly



(/wiki/File:Pico-

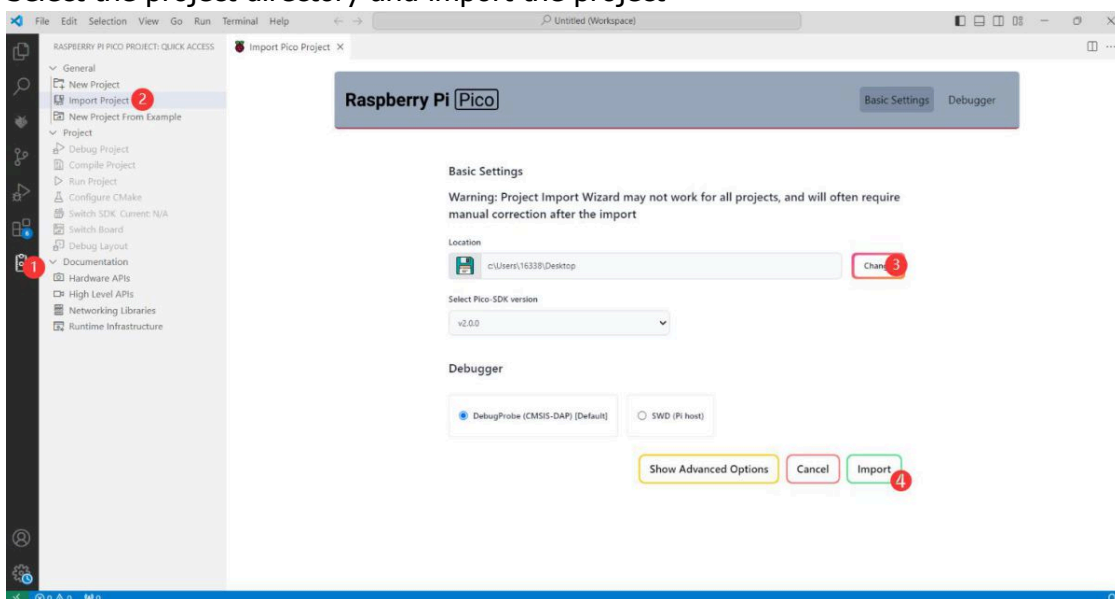
vscode-24.jpg)

2. Flash the firmware manually

1. Press and hold the Boot button
2. Connect the development board to the computer
3. Then the computer will recognize the development board as a USB device.
4. Copy the .uf2 file to the USB drive, and the device will automatically restart, indicating successful program flashing.

Import Project

1. Select the project directory and import the project

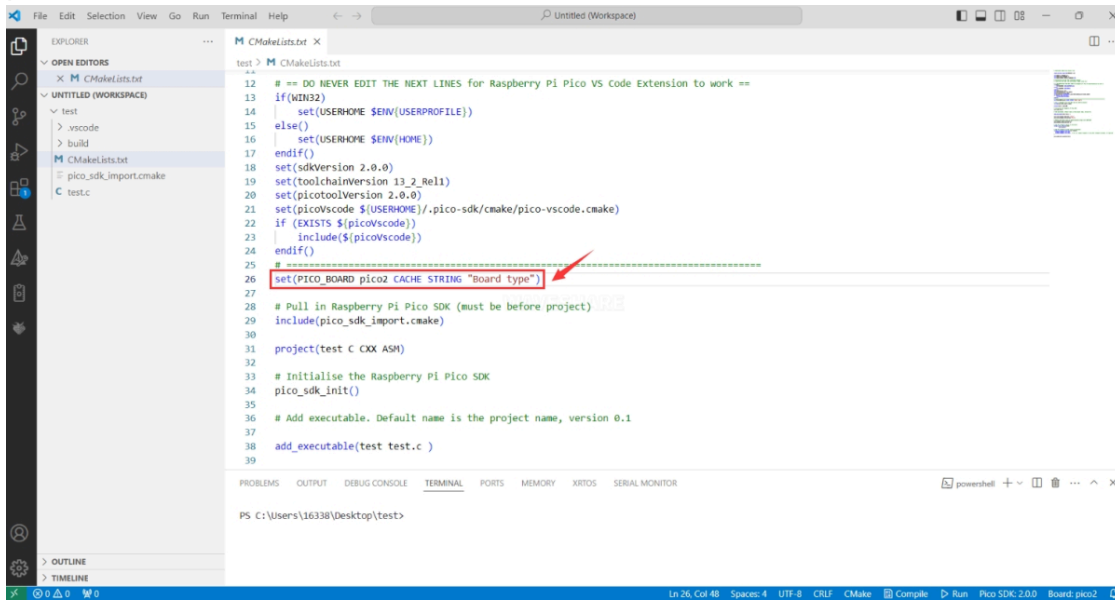


(/wiki/File:Pico-

vscode-23.jpg)

2. The Cmake file of the imported project cannot have Chinese (including comments), otherwise the import may fail

- To import your own project, you need to add a line of code to the Cmake file to switch between pico and pico2 normally, otherwise even if pico2 is selected, the compiled firmware will still be suitable for pico



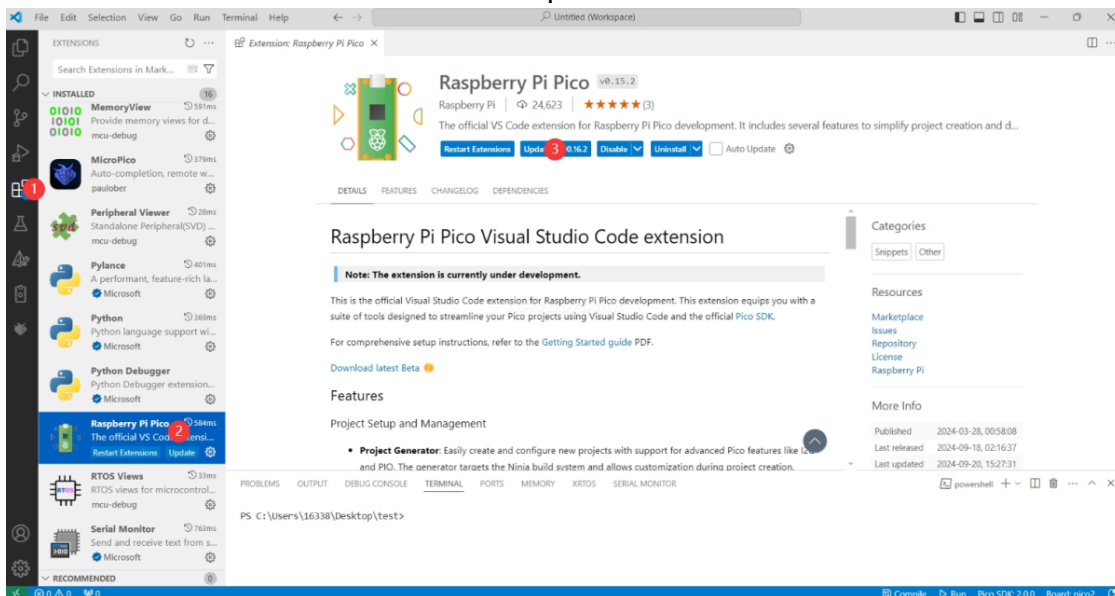
(/wiki/File:Pico-

vscode-21.png)

```
set(PICO_BOARD pico CACHE STRING "Board type")
```

Update Extension

- The extension version in the offline package is 0.15.2, and you can also choose to update to the latest version after the installation is complete



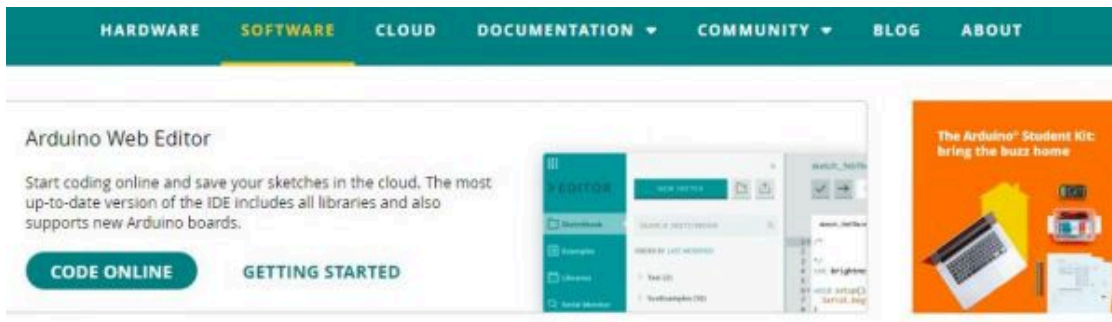
(/wiki/File:Pico-

vscode-22.png)

Arduino IDE Series

Install Arduino IDE

- First, go to Arduino official website (<https://www.arduino.cc/>) to download the installation package of the Arduino IDE.



Downloads



Arduino IDE 2.0.0

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file
Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)
macOS 10.14: "Mojave" or newer, 64 bits

(/wiki/File:600px-Arduino%E4%B8%8B%E8%BD%BD2.0%E7%89%88%E6%9C%AC.jpg)

2. Here, you can select Just Download.

Support the Arduino IDE

Since the release 1.x release in March 2015, the Arduino IDE has been downloaded **69,954,557** times — impressive! Help its development with a donation.

\$3

\$5

\$10

\$25

\$50

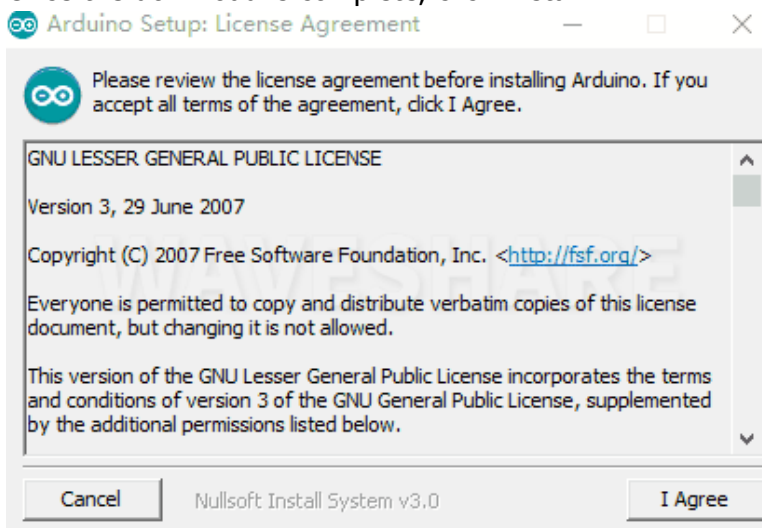
Other



Learn more about [donating to Arduino](#).

(/wiki/File:%E4%BB%85%E4%B8%8B%E8%BD%BD%E4%B8%8D%E6%8D%90%E8%B5%A0.png)

3. Once the download is complete, click Install.



(/wiki/File:IDE%E5%AE%89%E8%A3%85%E6%B0%B4%E5%8D%B0-1.gif)

Notice: During the installation process, it will prompt you to install the driver, just click Install

Arduino IDE Interface

1. After the first installation, when you open the Arduino IDE, it will be in English. You can switch to

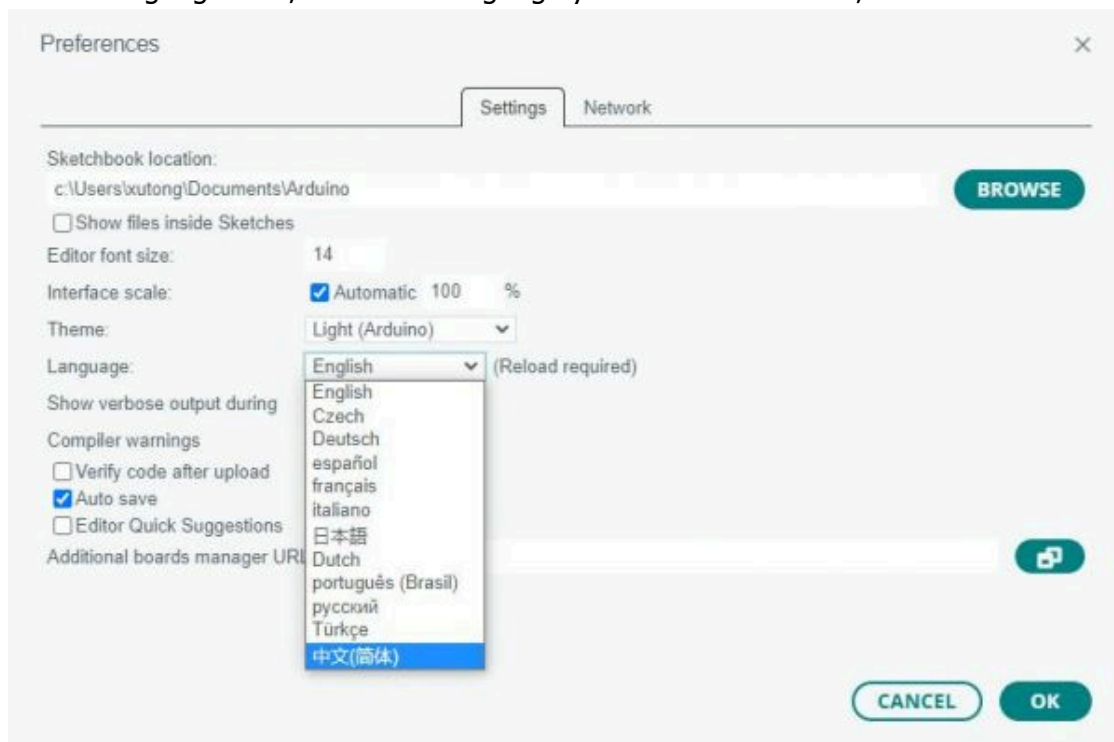
other languages in File --> Preferences, or continue using the English interface.



(/wiki/File:%E9%A6%96%E9%80%89%E9%A1%B9-

%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87.jpg)

2. In the Language field, select the language you want to switch to, and click OK.



(/wiki/File:600px-%E9%A6%96%E9%80%89%E9%A1%B9-

%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87ok.jpg)

Install Arduino-Pico Core in the Arduino IDE

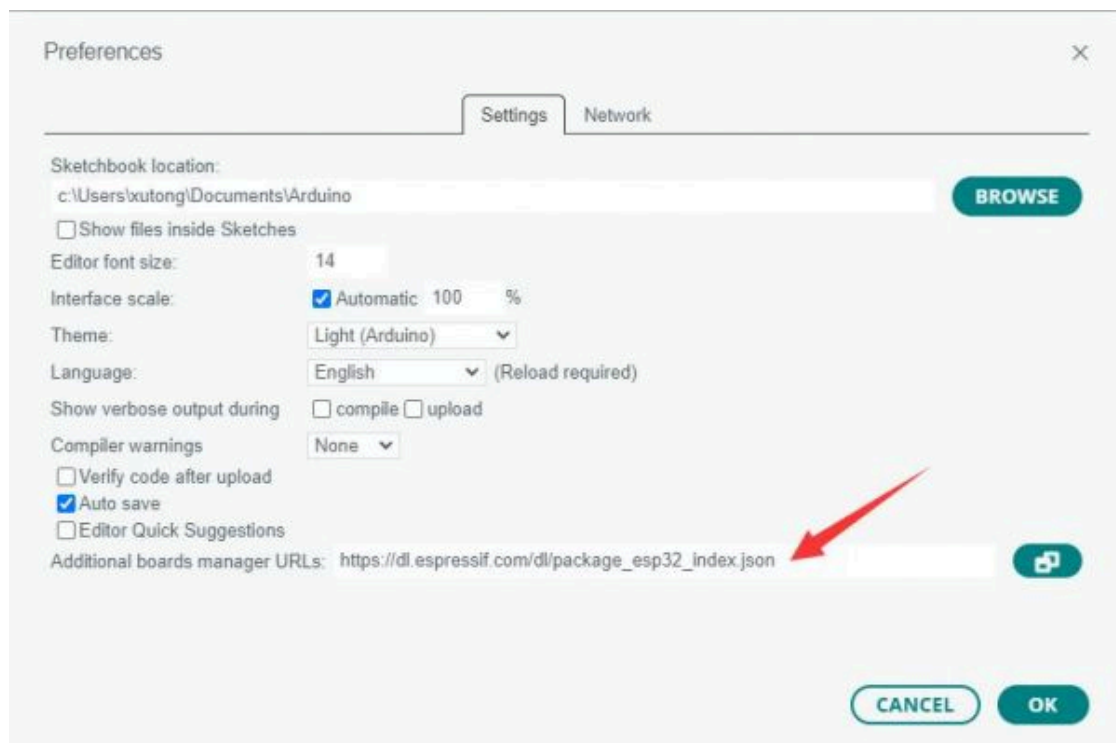
1. Open the Arduino IDE, click on the file in the top left corner, and select Preferences



(/wiki/File:RoArm-M1_Tutorial04.jpg)

2. Add the following link to the attached board manager URL, and then click OK

`https://github.com/earlephilhower/arduino-pico/releases/download/4.0.2/package_rp2040_index.json`

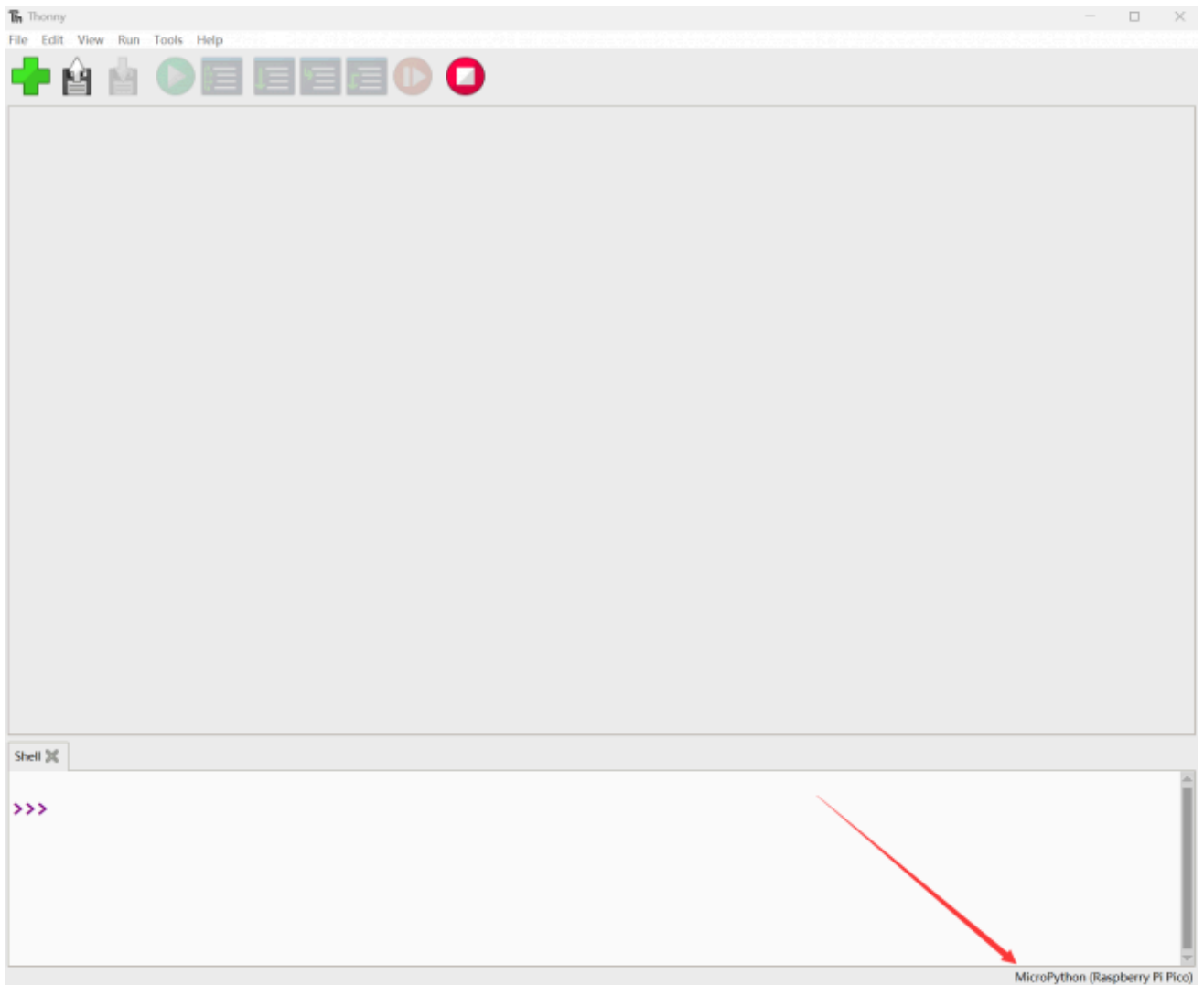


(/wiki/File:RoArm-M1_Tutorial_II05.jpg)

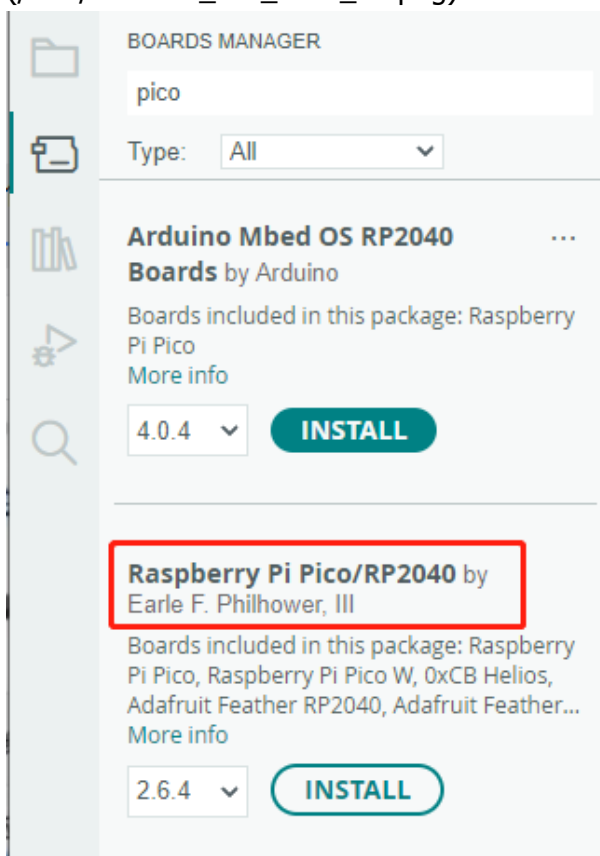
Note: If you already have an ESP32 board URL, you can use a comma to separate the URLs as follows:

`https://dl.espressif.com/dl/package_esp32_index.json,https://github.com/earlephilhower/arduino-pico/releases/download/4.0.2/package_rp2040_index.json`

3. Click Tools > Development Board > Board Manager > Search pico, as my computer has already been installed, it shows that it is installed



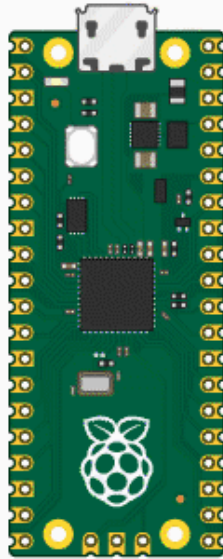
(/wiki/File:Pico_Get_Start_05.png)



(/wiki/File:Pico_Get_Start_06.png)

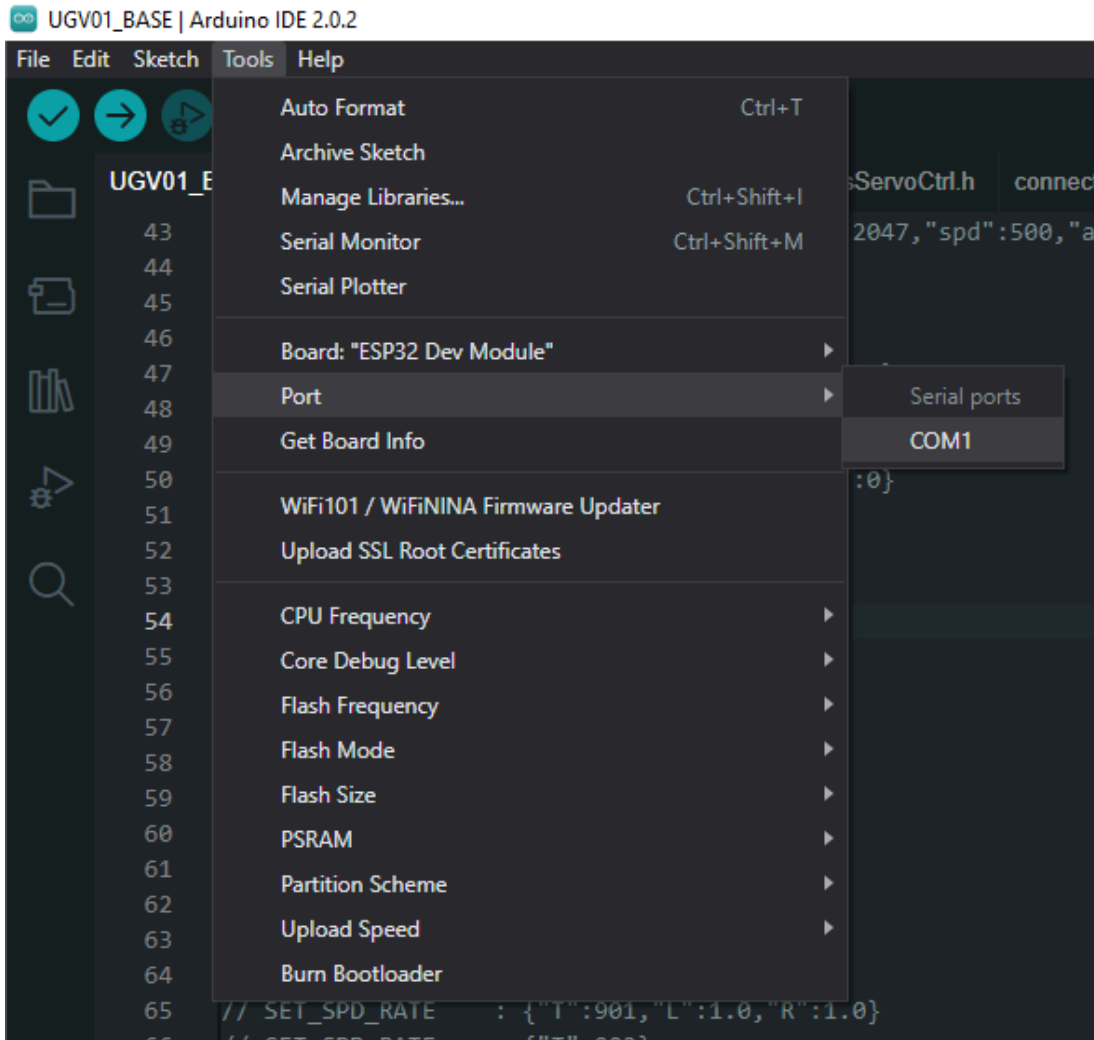
Upload Demo at the First Time

1. Press and hold the BOOTSET button on the Pico board, connect the pico to the USB port of the computer via the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).



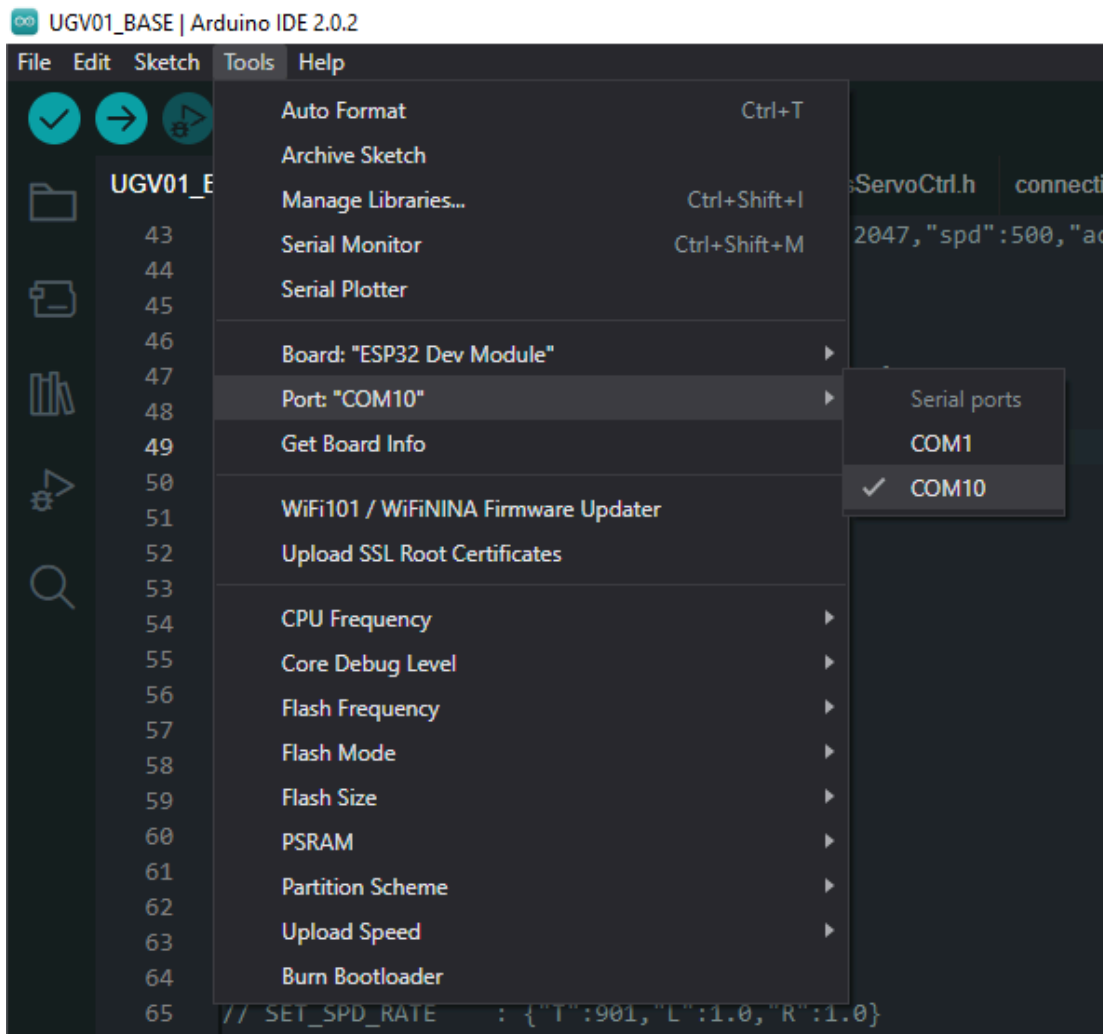
(/wiki/File:Pico_Get_Start.gif)

2. Download the program and open D1-LED.ino under the arduino\PWM\D1-LED path
3. Click Tools --> Port, remember the existing COM, do not click this COM (the COM displayed is different on different computers, remember the COM on your own computer)



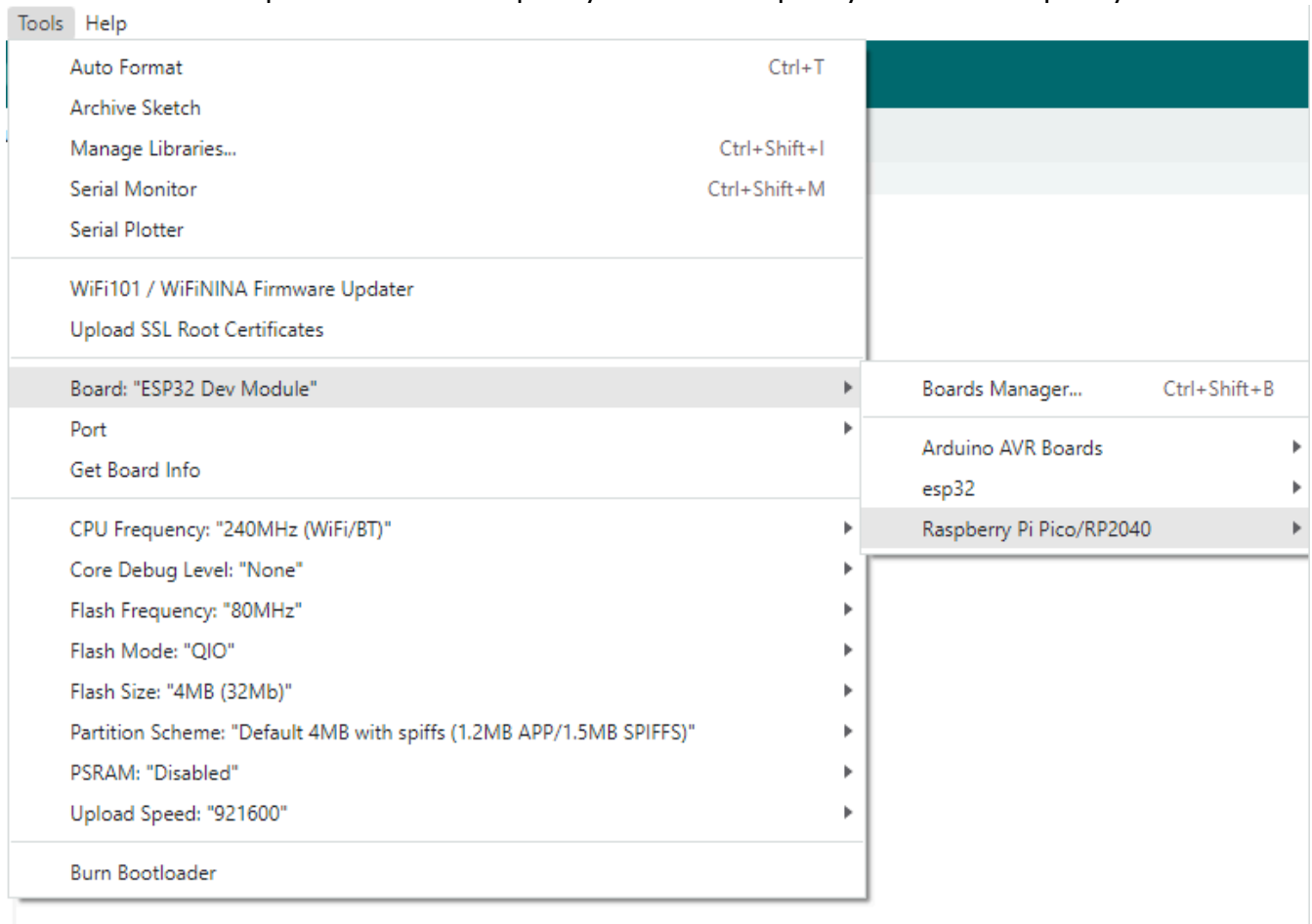
(/wiki/File:UGV1_download02EN.png)

4. Connect the driver board to the computer using a USB cable. Then, go to Tools > Port. For the first connection, select uf2 Board. After uploading, when you connect again, an additional COM port will appear



(/wiki/File:UGV1_download03EN.png)

5. Click Tools > Development Board > Raspberry Pi Pico > Raspberry Pi Pico or Raspberry Pi Pico 2



(/wiki/File:Pico_Get_Start02.png)

6. After setting it up, click the right arrow to upload the program

 sketch_aug16a | Arduino IDE 2.1.0

File Edit Sketch Tools Help



(/wiki/File:Pico_Get_Start03.png)

- **If issues arise during this period, and if you need to reinstall or update the Arduino IDE version, it is necessary to uninstall the Arduino IDE completely. After uninstalling the software, you need to manually delete all contents within the C:\Users\[name]\AppData\Local\Arduino15 folder (you need to show hidden files to see this folder). Then, proceed with a fresh installation.**

Open Source Demos

MircoPython video demo (github) (https://github.com/waveshareteam/Pico_MircoPython_Examples)

MicroPython firmware/Blink demos (C) (https://files.waveshare.com/wiki/common/Raspberry_Pi_Pico_Demo.zip)

Raspberry Pi official C/C++ demo (github) (<https://github.com/raspberrypi/pico-examples/>)

Raspberry Pi official micropython demo (github) (<https://github.com/raspberrypi/pico-micropython-examples>)

Arduino official C/C++ demo (github) (<https://github.com/earlephilhower/arduino-pico>)

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the issue.

Working Time: 9 AM - 6 PM GMT+8 (Monday to Friday)

Submit Now (<https://service.waveshare.com/>)

Retrieved from "<https://www.waveshare.com/w/index.php?title=Pico-RGB-LED&oldid=91346>
(<https://www.waveshare.com/w/index.php?title=Pico-RGB-LED&oldid=91346>)"